

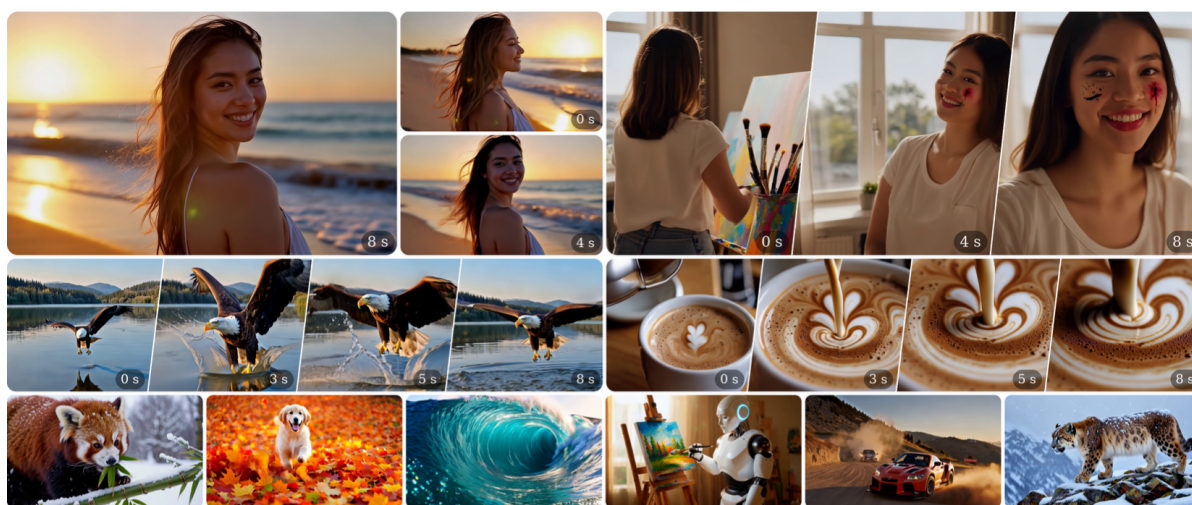
SANA-Video 2.0: Hybrid Linear Attention with Attention Residuals for Efficient Video Generation

Junsong Chen^{1,2}, Jincheng Yu^{1,2}, Yitong Li^{1,2}, Shuchen Xue¹, Haozhe Liu¹, Jingyu Xin¹
Yuyang Zhao¹, Zhangjie Wu¹, Zian Wang¹, Daquan Zhou¹, Ping Luo², Enze Xie¹, Song Han^{1,3}

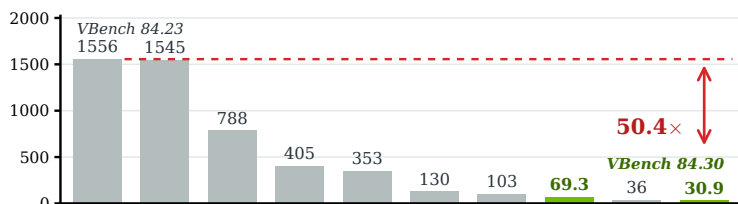
¹NVIDIA Research, Efficient AI Team & Singapore Lab ²HKU ³MIT

[Project Page](#) [GitHub](#)

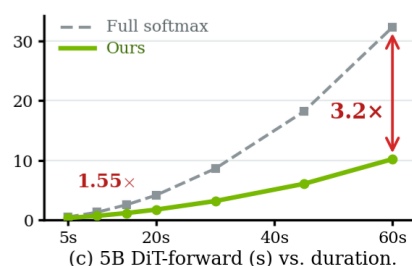
Abstract: We introduce SANA-Video 2.0, a mostly-linear video diffusion transformer instantiated at 5B and 14B scales under one shared architecture. The design generates high-quality video up to 720p on a single GPU, matching strong full-softmax Video DiTs in quality while retaining the favorable long-sequence scaling of linear attention. To avoid quadratic attention throughout, **Hybrid Linear-Softmax Attention** combines gated linear attention for $O(N)$ -dominated mixing with periodic gated-softmax anchors at a regular 3:1 ratio (25% softmax), restoring the full-rank token interactions that pure linear attention cannot express. To propagate these refreshed representations across depth, **Block Attention Residuals (AttnRes)** route completed block summaries into later linear layers, enabling anchor-feature reuse and increasing deep-layer effective rank by $\sim 12\%$. Through **from-scratch training**, we learn the complete hybrid from random initialization rather than linearizing a pretrained softmax model or initializing from an image model, and reduced-resolution proxy studies identify 25% softmax as the quality–efficiency knee. With 40-step sampling, SANA-Video 2.0 reaches VBench Total 84.30 in 13.2 s at $480 \times 832 \times 81$ on one H100, competitive with far larger softmax Video DiTs at a fraction of the latency. Its compiled DiT forward is **3.2 $\times$** faster than matched full softmax at a 720p/60s shape, an advantage that widens with duration. In a separate 50-step B200 deployment study, the full stack reduces end-to-end latency by **3.58 $\times$** . These results show that a scratch-trained, mostly-linear hybrid can recover softmax-level expressiveness at substantially lower cost while scaling efficiently to long, high-resolution video.



(a) 720p text-to-video samples, up to 8 s; grouped frames unroll one clip over time.



(b) Generation-pipeline latency (s), one H100 (720p, 81f).



(c) 5B DiT-forward (s) vs. duration.

Figure 1 | **SANA-Video 2.0 at a glance.** (a) 720p text-to-video samples. (b) One-H100 latency at 720p/81f; VBench Total marks the 5B and Wan 2.2 quality points, and SANA-Video 2.0 bars use 40 steps (14B: registered config, random weights). (c) 5B DiT-forward speedup over 720p duration; protocols in Appendix C.2.

1. Introduction

Video generation is a rapidly advancing field, powering applications from creative content production to virtual product displays and live streaming. Recent large-scale video diffusion transformers (Video DiTs) [24], such as Wan 2.1 [34] (14B), HunyuanVideo [14] (13B), and Seedance 2.0 [29], alongside closed systems such as Veo 3 and Kling, produce remarkably high-fidelity video. Yet this quality rests on full 3D softmax attention, whose $O(N^2)$ cost is punishing for video: after VAE compression a single 1080p clip already spans tens of thousands of latent tokens, and in our matched production-scale profiling sweep a full-softmax backbone is $2.01\times$ slower than our 25% hybrid at 1080p/121f, with both sides compiled on their best kernels (Figure 5(a)). The problem worsens for long video (>10 s), where full-sequence softmax and its growing KV cache dominate the cost, and even dedicated long-video efforts such as MAGI-1 [28] remain constrained by vanilla attention.

Linear attention offers a principled escape. Its $O(N)$ complexity scales gracefully with sequence length, and its state-space view exposes a clean interface for recurrent kernels, and SANA-Video [3] showed that a *pure*-linear Video DiT already delivers large speedups at competitive quality. But pure linear attention pays for its efficiency with expressiveness: the fixed-size state matrix $S \in \mathbb{R}^{d \times d}$ cannot represent every token–token interaction, blurring precise spatiotemporal correspondence and fine detail [1, 4]. Recent large language models point to a fix, which is to keep a mostly linear-attention stack but insert a few softmax *anchors*—periodic full-attention layers that re-establish, at fixed depths, the global full-rank token interactions linear attention loses—at a regular ratio (Qwen3-Next [27], Kimi-Linear [11]), and route information across depth with Attention Residuals [13], a combination the recent Kimi K3 [12] adopts at trillion-parameter scale. This raises a central question: *can a mostly-linear backbone recover the expressiveness of softmax attention while keeping its $O(N)$ scaling for long, high-resolution video?*

This paper answers affirmatively with SANA-Video 2.0, a hybrid-attention video diffusion transformer with 5B and 14B configurations that share the same architecture and training method. The 5B configuration keeps a mostly-linear backbone but inserts a small number of softmax anchors, matching strong full-softmax Video DiTs in quality while remaining fast on a single GPU (Figure 1): with 40-step sampling it reaches VBench Total 84.30 in 13.2 s at $480 \times 832 \times 81$ on one H100, competitive with much larger softmax models at a fraction of their latency, and its DiT forward is $3.2\times$ faster than a matched full-softmax DiT at a 720p/60 s shape with both compiled on their best kernels (Figure 1(c)), an advantage that widens with duration. Unlike post-hoc linearization of a pretrained softmax model, we train the hybrid backbone from random initialization. The design rests on three key components.

Hybrid Linear-Softmax Attention. We keep gated bilinear linear attention as the core token-mixing operation for its $O(N)$ cost, and interleave gated softmax anchors at a regular 3:1 ratio (25% softmax) placed at fixed depths. The anchors periodically restore the less rank-constrained token interactions that pure linear attention cannot express, while the linear majority preserves long-sequence scaling. Text enters through cross-attention, and a convolution-free SwiGLU FFN replaces SANA-Video’s temporal-convolution FFN, whose measured overhead grows with duration (Appendix F.6), keeping the backbone easy to profile and fuse with off-the-shelf kernels.

Block Attention Residuals (AttnRes). Additive residuals alone force later layers to re-derive information computed upstream. Instead of this re-derivation, AttnRes routes completed block-feature summaries across depth, so later linear layers can reuse the anchors’ rank-refreshed updates. In a controlled same-checkpoint on/off probe, enabling AttnRes raises deep-layer state effective rank by $\sim 12\%$. We adapt the routing to bidirectional video diffusion by sharing routing queries across depth and removing explicit diffusion-timestep conditioning, made redundant by AdaLN.

From-Scratch Training and Design. We select the architecture with short, reduced-resolution proxy studies that identify 25% softmax anchors as a practical quality–efficiency knee. The full model is then trained from scratch through a complete, documented multi-stage pipeline, comprising diverse-source data curation and filtering, a low-to-high resolution and duration curriculum, structured captioning, and self-distillation, followed by preference-based post-training (DPO and ReFL), so the hybrid learns motion and appearance without relying on a pretrained softmax or image prior. We report this training and inference-optimization pipeline in full as a central, reproducible contribution.

In conclusion, SANA-Video 2.0 attains competitive VBench quality (84.30) at markedly lower latency than large softmax Video DiTs, and its $O(N)$ -dominated backbone scales far better with duration: $3.2\times$ faster per DiT forward than full softmax at 720p/60 s under matched best-kernel compilation. A separate 50-step Sol-Engine deployment directly measures a $3.58\times$ end-to-end speedup on B200 (Appendix F.10). Rank and routing analyses reveal a clear division of labor, where linear layers provide inexpensive global mixing, softmax anchors inject the missing full-rank interactions, and AttnRes carries block-level features across depth. We hope SANA-Video 2.0 offers a practical, efficient foundation for high-quality video generation that researchers and everyday users can run fast.

2. Preliminaries

Video DiTs and flow matching. Video Diffusion Transformers [24] denoise a latent sequence $z_t = (1-t)z + t\epsilon$ ($\epsilon \sim \mathcal{N}(0, I)$, $t \sim \text{logit-normal}$) by predicting the velocity $v_\theta \approx \epsilon - z$ with loss $\mathcal{L} = \mathbb{E}\|v_\theta - (\epsilon - z)\|^2$. Here $t \in [0, 1]$ denotes the normalized flow-matching timestep. Each block uses AdaLN-style timestep modulation, self-attention, text cross-attention, and a FFN. At 1080p a long clip already spans tens of thousands of latent sites, making softmax’s $O(N^2)$ cost prohibitive.

Gated linear attention. Linear attention compresses token interactions into a fixed-size state, reducing sequence complexity to $O(Nd^2)$ at the cost of a rank bottleneck. Our bidirectional operator is a gated bilinear form rather than a causal delta-rule recurrence; dropping the delta-rule update also makes it a natural initialization for a causal Gated DeltaNet, a direction we leave to future work. For normalized queries and keys, let q_j^r, k_n^r denote their RoPE-rotated forms and $\phi(\cdot) = \text{ReLU}(\cdot)$. Each head computes

$$S = \sum_n v_n (\beta_n k_n^r)^\top, \quad o_j = W_o \left[\text{RMSNorm} \left(\frac{S q_j^r}{\sum_n \phi(k_n^r)^\top \phi(q_j) + \epsilon} \right) \odot \sigma(g_j) \right], \quad (1)$$

where β gates writes to the state and g gates the output. Periodic softmax layers provide less rank-constrained token mixing at selected depths while leaving most sequence computation linear. These anchors use bidirectional SDPA with QK normalization, RoPE, and the sigmoid output gate studied by Qiu et al. [26]. We adopt the regular 3:1 hybrid layout of Qwen3-Next [27]; Kimi-Linear [11] independently uses the same proportion with a different linear operator.

Attention Residuals. Standard additive residuals $h_l = h_{l-1} + f_l(h_{l-1})$ propagate information through every intervening depth. Attention Residuals [13] instead use learned depth-wise aggregation, $h_l = \sum_i \alpha_{i \rightarrow l} v_i$. Their Block AttnRes variant keeps completed block summaries and a running within-block residual, and applies separate routing before attention and FFN sublayers. In our hybrid stack, completed summaries include softmax-anchor updates and expose them to later mostly-linear layers. The original parameterizes routing queries per sublayer. Our video adaptation instead shares them across depth. Rather than assume the routing queries need a separate diffusion-timestep input, we test whether one is necessary and find it redundant with AdaLN.

3. SANA-Video 2.0

3.1. Overview

SANA-Video 2.0 is a video DiT that combines hybrid sequence attention with block residual attention across depth. It operates on LTX-VAE 2.3 [8] latents and draws text features from Gemma-2-2B-IT [5] through cross-attention at every layer. At each depth, AttnRes first routes a representation into the self-/cross-attention branch and, in a second step, into the SwiGLU FFN, with AdaLN-style modulation applied inside each branch, and a final depth aggregation precedes the output head. The local path is convolution-free throughout. Appendix A lists the full configuration.

3.2. Hybrid Attention Design

Building on the gated attention primitives (Section 2), we interleave bidirectional gated linear attention layers with gated softmax anchors at a 3:1 ratio, 75% linear, 25% softmax, placing a softmax anchor at every fourth layer. The linear majority keeps token mixing at $O(N)$, while the uniformly spaced anchors periodically refresh the less rank-constrained interactions that the compressed linear state cannot represent (Section 5.5). Linear and softmax heads use different head dimensions, trading efficiency against per-head capacity (Appendix A). This regular layout follows the hybrid linear attention of recent LLMs (Qwen3-Next [27], Kimi-Linear [11], and Kimi K3 [12] at trillion-parameter scale) and maps cleanly onto fused linear-attention kernels. We confirm the 25% anchor ratio as a quality–efficiency knee for the video regime by sweeping it from scratch rather than assuming the language-model value (Section 5.3). The two paths retain separate RoPE [31] tensors and gating parameterizations: linear layers use both the write gate β and an output gate, whereas softmax anchors use the sigmoid output gate of Qiu et al. [26].

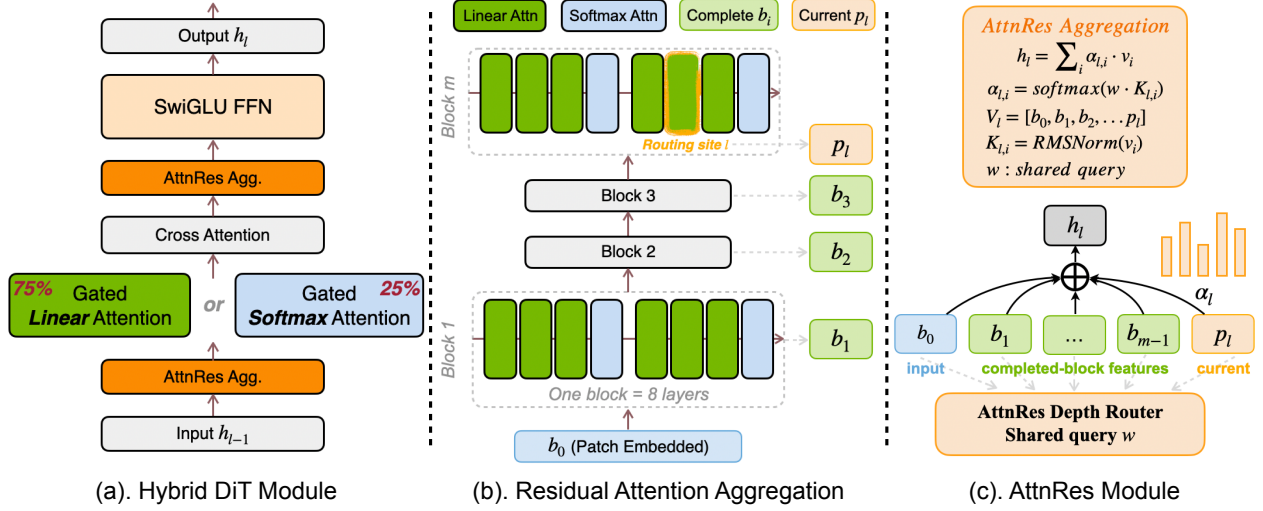


Figure 2 | **SANA-Video 2.0 overview.** (a) Hybrid DiT layer. (b) Eight-layer blocks expose completed features across depth. (c) A shared-query AttnRes router aggregates the input and completed block features.

3.3. Block Attention Residuals (AttnRes)

Figure 2 summarizes the hybrid layer and its block-level depth routing. Hybrid attention supplies less rank-constrained token mixing only at sparse depths; AttnRes complements it by exposing completed block summaries containing those updates to later layers. We build on the Block AttnRes [13], which Kimi K3 [12] pairs with hybrid linear attention at language-model scale, and adapt it to bidirectional video diffusion. The layers are grouped into consecutive blocks: each block that has finished by layer l leaves a single feature summary, and the block still in progress carries a running partial sum. The router at layer l therefore draws on a source set $\mathcal{V}_l = \{b_0, b_1, \dots, p_l\}$ made of the initial token embedding b_0 , the completed block summaries $b_1, b_2, \dots$ from blocks that finished before layer l , and the current within-block partial sum p_l , the accumulation so far in the in-progress block (absent only at a block boundary, where it has just been frozen into a new summary and reset). For each video token x , the routed representation before sublayer type τ is

$$h_l(x) = \sum_{v_i \in \mathcal{V}_l} \alpha_{i \rightarrow l}^{(\tau)}(x) v_i(x), \quad \alpha_{i \rightarrow l}^{(\tau)}(x) = \text{softmax}_i \left((w^{(\tau)} + \phi_\tau(t))^\top \text{RMSNorm}(v_i(x)) \right), \quad (2)$$

where $v_i(x)$ is the feature that source $v_i \in \mathcal{V}_l$ holds at token position x , $w^{(\tau)}$ is shared by all depths for $\tau \in \{\text{attn}, \text{ffn}\}$, and $\phi_\tau(t)$ is an optional, zero-initialized timestep offset. The softmax is over depth sources independently for every token. After the last block, a third shared query aggregates the initial embedding and completed block sums before the output head.

Shared routing queries. The original AttnRes learns a separate routing query for every residual sublayer, a per-layer design that language models keep even at scale (e.g., Kimi K3 [12]). For our video model, one shared routing query per branch suffices—a single attention query and a single FFN query, reused at every depth—which matches per-layer routing in loss at a fraction of the memory (Section 5.3). Sharing the query does not make routing uniform. The source set $\mathcal{V}_l$ changes with depth, so the routing weights still vary from layer to layer, and because each source is normalized per token, they also vary from token to token. Sharing thus drops the per-depth query parameters while still routing the attention and FFN branches differently.

Block organization. We group every $S=8$ consecutive transformer layers into one block. As the forward pass moves through a block, each attention or FFN sublayer’s output—the residual update it would otherwise add to the hidden state—is added into a running partial sum p_l , which the router also reads back as a source (Eq. 2). When the block ends, p_l is frozen as that block’s completed summary b_k , and the next block starts a fresh partial sum. Because the router keeps only one summary per finished block rather than one feature per layer, the stored history shrinks from $O(NLd)$ to $O(N\lceil L/S \rceil d)$ for sequence length N , roughly an S -fold reduction. The span of $S=8$ covers two softmax-anchor

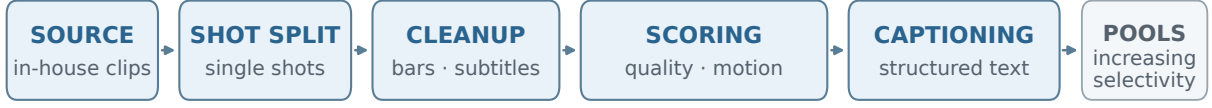
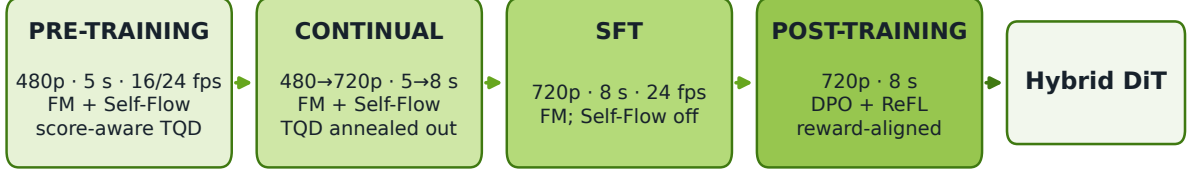
IN-HOUSE DATA PROCESSING**TRAINING CURRICULUM** *continuous run · resolution · duration · quality ↑*

Figure 3 | **Training and data pipeline.** Curated in-house clips feed pre-training, continual training, SFT, and preference-based post-training (DPO + ReFL), with stage-specific data selection, video shapes, and training objectives per stage.

cycles while keeping this history compact, and the ablation treats it as an engineering default rather than a universal optimum (Section 5.3). As in Block AttnRes, the attention and FFN routers stay separate and learn different depth preferences (Section E.5).

Timestep-independent final design. Although denoising timesteps have different semantic roles, the learned offset is nearly constant and changing or shuffling it has negligible validation effect. The final model therefore uses $\phi_\tau \equiv 0$ while retaining timestep modulation in the DiT blocks (Section 5.3, Appendix E.4).

4. Training Recipe

SANA-Video 2.0 is instantiated in 5B and 14B configurations of the same hybrid backbone. The 5B configuration follows a staged continuation from scratch, spanning data curation, a resolution/duration curriculum, and preference-based post-training (Figure 3). Beyond the architecture, its training recipe specifies how the corpus is transformed into progressively cleaner supervision, how noise levels are sampled, and how resolution and duration are increased. We describe those components because they are part of the method. The recipe was selected jointly rather than component-wise ablated. Appendix A lists the architecture and hyperparameters of both configurations.

4.1. Data Curation and a Quality/Motion Funnel

Starting from the raw video corpus, we apply a uniform processing pipeline: shot segmentation, black-bar and subtitle cleanup, removal of low-resolution/blurry/static/exposure-defective clips, and multi-axis scoring rather than a single aggregate quality score. Keeping the axes separate is deliberate: it lets the funnel retain both motion-rich clips and visually strong low-motion clips, so a quality filter never collapses the corpus to fast-motion content alone. Appendix B gives the complete six-stage pipeline and scoring signals (Table 5). The resulting clips form a progressive quality/motion funnel: pre-training uses a broad pool under permissive thresholds, continual training raises the quality and motion floors, and SFT concentrates on a small trusted subset. Captions progress from mixed short/long descriptions to dense

Table 1 | **5B production training curriculum.** Clip counts are reported after filtering; objective add-ons are stage-specific. The 14B hyperparameters are listed in Table 4.

Stage	Clips	Resolution	FPS	Duration	Flow-shift	LR	Objective add-on
Pre-train	~30M	480p	16/24	5s	1 (+TQD)	10^{-4}	TQD + Self-Flow
Continual	~10M	480→720p	24	5s→8s	3–6 (by tokens)	10^{-4}	Self-Flow
SFT	~ 10^4	720p	24	8s	3–6 (by tokens)	5×10^{-5}	standard FM

structured ones, and quantized motion descriptors are appended when available to turn the selection signal into a conditioning cue. The same quality and motion axes also drive the pre-training timestep policy below, establishing broad coverage early and focusing later stages on fidelity and prompt adherence.

4.2. Objective and Resolution-Aware Timestep Sampling

Training uses flow matching with a logit-normal timestep density. In later continual and SFT stages we tail-floor this density, reserving probability mass near the clean and pure-noise extremes that a plain logit-normal undersamples. These regimes respectively correspond to final refinement and global structure. From the continual stage onward we make flow-shift token-count-aware: rather than a fixed per-resolution constant, we anchor the shift at two endpoints and interpolate in log-shift space by latent token count, running log-linearly from 3 at 4,290 latent tokens (480p, 81 frames) to 6 at 23,000 tokens (720p, 193 frames). Each shift is realized by offsetting the (tail-floored, $\sigma=0.95$) logit-normal mean by $\log(\text{shift})$, so every intermediate resolution $\times$ duration lands on a principled shift and more probability mass moves to high noise as clips lengthen (Table 1). Because our curriculum varies both resolution and duration, the token count sweeps a wide continuous range that a discrete per-resolution setting would not cover. The mapping is applied per sample in mixed-resolution batches.

Bucketed validation monitoring. Because loss varies several-fold across the noise axis, the usual random- t mean can hide offsetting checkpoint changes even though it remains an unbiased aggregate objective estimate. We therefore monitor conditional validation means in ten equal 100-step noise buckets. Across four paired checkpoints, the bucket macro falls by 6.42%, but the profile separates an 11.44% low-noise improvement from a 1.16% high-noise regression (Fig. 8). Balancing the 100 evaluations across buckets also cuts the variance of this early-to-final estimate by $2.27\times$ relative to IID timestep draws, and VBench Total rises from 82.68 to 83.29 over the same four checkpoints. Appendix A.4 details the matched-sample protocol and uncertainty analysis.

Content-aware flow-shift (TQD, pre-training only). High-quality video data is scarce, so we want each clip to contribute where it teaches best. This runs into a motion-quality dilemma: motion-rich clips often carry artifacts that lower their per-frame aesthetics, whereas the cleanest clips tend to be nearly static, so a single quality bar sacrifices either motion or fidelity. TQD [23] resolves this by routing each clip to the noise regime where its content matters and its weaknesses do not: high noise governs global structure and motion and masks fine texture, so it receives the high-motion clips, while low noise governs fine detail and receives the aesthetically clean, typically low-motion clips. In pre-training we realize this by reusing the per-clip motion and quality scores from filtering to bias high-motion clips toward high noise and high-quality clips toward low noise, so every clip trains mostly at the timesteps it can teach best. From continual training onward the token-count shift takes over and TQD is annealed out with no discontinuity in the noise schedule. Appendix A.3 gives the bias magnitudes, thresholds, the handoff, and full sampling densities.

Weight EMA and Self-Flow. We keep an exponential moving average (EMA) of the model weights active throughout training, including SFT. Alongside it, pre-training and continual training additionally carry Self-Flow, an auxiliary feature-distillation objective with within-clip dual-timestep conditioning that we use as a training aid rather than a studied component. Self-Flow leaves every latent token supervised, and it is turned off for SFT while the weight EMA stays on (Appendix A.2).

4.3. Multi-Resolution, FPS, and Duration Curriculum

The run advances resolution (480p $\rightarrow$ 720p), frame rate, and duration together, with the 720p share increasing in later stages (Table 1). Frame counts are chosen so each clip lands on an integer number of LTX-VAE latent frames. Because FSDP runs at per-rank batch size one, the sampler buckets each rank into a single (aspect-ratio, frame-count) tier per step, so resolution/duration mixing does not desynchronize ranks. Image batches are interleaved with video at a fixed cadence to inject appearance quality into a motion-biased corpus.

4.4. Post-Training Stage

4.4.1. Direct Preference Optimization

We employ Diffusion-DPO [33] to further align the model with human visual preferences. For each text prompt, we sample multiple videos from our model using different random seeds. We then use Gemini as an automatic evaluator to rank the generated videos according to their overall quality, including visual fidelity, prompt alignment, and temporal coherence. The highest-ranked and lowest-ranked videos are selected as the preferred and rejected samples, respectively, forming a preference pair for the corresponding prompt. These automatically constructed preference pairs are subsequently used to optimize the model with the Diffusion-DPO objective.

4.4.2. Online Reinforcement Learning

We further align the model through online reward optimization using Reward Feedback Learning (ReFL) [38]. For each text prompt, ReFL first rolls out the denoising trajectory without gradients to a randomly sampled update step. At that state, our diffusion model predicts the clean latent $\hat{x}_0$, which is decoded into a video and scored by frozen reward models. Gradients from the reward are propagated through the decoder and the model evaluation, avoiding backpropagation through the complete sampling trajectory.

Our joint reward combines HPSv3++ [21], DeQA-Score [41], and UniPercept [2]. The three models provide complementary supervision. HPSv3++ targets wide-spectrum human preference alignment across the capability-iteration spectrum, with particular emphasis on *text fidelity* and *aesthetic quality*. DeQA-Score provides a no-reference perceptual-quality signal: it models continuous human quality judgments through a predicted *score distribution*, making it sensitive to fidelity loss and visible image degradations. UniPercept supplies a finer-grained perceptual profile spanning Image Aesthetics Assessment (IAA), Image Quality Assessment (IQA), and Image Structure and Texture Assessment (ISTA). We combine the normalized and capped outputs into a weighted joint reward. We also add a velocity-space mean-squared-error regularizer against the frozen base model. The resulting objective improves prompt-conditioned human preference and frame-level perceptual quality while preserving the generative prior acquired during pre-training and supervised fine-tuning.

5. Experiments

5.1. Implementation Details

SANA-Video 2.0 has 5B and 14B configurations of the same hybrid backbone. Table 4 lists both in full. The checkpoint-quality experiments use the 5B configuration, which starts from random initialization and is continued with AttnRes during later training. The 14B measurements are systems-only because no trained 14B checkpoint is currently available. The 5B model uses a curated video corpus, LTX-VAE 2.3 latents, Gemma text features, flow matching, and a 480p–720p curriculum on 64×H100 GPUs, with the recipe in Section 4. Our design studies are intentionally short and reduced-resolution: the attention-ratio and AttnRes span sweeps use 5B depth-28 proxies at 256p, while the query/timestep ablations and mechanism probes use 5B architecture-family checkpoints (all on 8×H100 GPUs). These studies select the architecture and provide mechanism diagnostics. They are not estimates of the production configuration’s final quality, and cross-family results are compared only within their stated protocol.

5.2. Main Results

Table 2 compares SANA-Video 2.0 with state-of-the-art video generators on VBench [9], while Figure 1(b) compares complete one-H100 generation-pipeline latency, including both our 5B checkpoint and formal 14B configuration under the same 40-step recipe. The 14B point is configuration-level latency from random weights and is kept separate from the 5B quality result.

At its 480×832×81 VBench operating point, SANA-Video 2.0 reaches Total 84.30 and the best Quality in Table 2 (85.61). The author-reported Bernini-R 14B [20] leads on Total and Semantic (84.64/82.49 vs. 84.30/79.05), while our hybrid leads on Quality. SANA-Video 2.0 is far cheaper to run: its 40-step pipeline latency is 13.2 s on one H100, against 421 s for Bernini-R under the same shape, step count, and device, a 31.8× gap (protocol in Appendix C). The 121- and 193-frame configurations reach Total 85.29 and 84.48 (Appendix Table 6), with the 121-frame model timed at 18.9 s. At a larger 720p-class shape (736×1280×81, same 40 steps), the 5B checkpoint takes 30.9 s. There, Cosmos-3

Table 2 | **VBench quality comparison.** Green intensity ranks the top three results per metric. Only the 81-frame SANA-Video 2.0 result is shown here; the full protocols and extended baselines are in Appendix C, which also lists our 121- and 193-frame operating points.

Model	Params	Attention	Frames	Total↑	Quality↑	Semantic↑
LTX-2.3 (base) [†]	22B	Full 3D	121	81.95	83.85	74.35
Cosmos-3 Nano [†]	16B	Full 3D	93	83.13	84.29	78.50
Lance [†]	7.1B	MoT	81	83.18	84.09	79.55
Wan 2.1	1.3B	Full 3D	81	83.31	85.23	75.65
HunyuanVideo*	13B	Full 3D	129	83.43	85.07	76.88
Wan 2.1	14B	Full 3D	81	83.69	85.59	76.11
SANA-Video	2B	Linear	81	84.17	84.85	81.46
Wan 2.2*	A14B	MoE	81	84.23	85.42	79.50
Bernini-R [‡]	14B	MoE	81	84.64	85.18	82.49
SANA-Video 2.0	5B	Hybrid	81	84.30	85.61	79.05

Nano and LTX-2.3 run $3.3\times$ and $4.2\times$ as long at their native shapes, and Bernini-R and Wan 2.2-A14B roughly $50\times$. The formal 48-layer, width-3,584 14B configuration takes 69.3 s at that same shape and recipe, $22.4\times$ faster than the 40-step Wan 2.2-A14B reference. At $480\times 832\times 81$ it takes 29.1 s, $14.5\times$ faster than matched-shape, 40-step Bernini-R. These 14B numbers establish systems feasibility only: random weights preserve the executed graph and latency but do not support a quality claim. For reference, SANA-Video-2B officially reports 36 s at 720p [3]. Full dimension-level scores, evaluation shapes, and timing protocols are in Appendix C, alongside the complete 16-dimension breakdown.

5.3. Design Space Exploration

We first explore the sequence-attention ratio with the 5B depth-28 proxies, trained from random initialization at 256p/81 frames and sharing data, hyperparameters, seed, and recipe across variants; we report training curves and validation loss on 200 held-out videos with text conditioning at 10K steps. Because validation loss is strongly non-uniform across the noise axis (Figure 4a), we read every proxy comparison per timestep rather than as a single scalar. The subsequent AttnRes probes follow the per-panel protocols in Table 3.

Axis 1: Softmax ratio. We train five variants spanning 0% (all-linear) to 100% (all-softmax), with three hybrid ratios in between (Figure 4). The lowest-ratio hybrid uses four softmax layers in the 28-layer proxy ($4/28=14.3\%$); “anchor” refers to one such periodic softmax layer.

Finding: Hybrid ratios perform better than the all-linear and all-softmax endpoints in this matched proxy search. Pure linear attention is the weakest endpoint (all-linear val loss 0.955), consistent with the absence of periodic softmax correction. All-softmax is also worse than the hybrids (0.945), with the two endpoints close and the linear one slightly worse. Among hybrids, 50% achieves the lowest loss (0.897) but at steep efficiency cost ($1.29\times$ latency over 25% hybrid at 1080p/121f, Appendix Table 9), while 25% (0.905) and the four-anchor (14.3%) hybrid (0.914) trail by only $\sim 1\%$. Together, Figure 4 and Appendix Table 9 identify 25% softmax as a practical Pareto knee rather than the minimum-loss point. Per-timestep analysis shows the hybrid-endpoint differentiation concentrates at mid noise ($t \approx 0.2-0.5$).

Lock-in: 25% softmax (3:1 ratio) for all subsequent experiments. This is the measured proxy Pareto knee, aligns with AttnRes block boundaries ($S=8$), and matches the Qwen3-Next layout [27]; Kimi-Linear [11] independently uses the same proportion with a different operator.

Scratch-training stability. All curves in Figure 4b start from random initialization; the hybrid variants converge smoothly with no loss spike or collapse and continue into the production training curriculum. The 3:1 ratio is thus trained from scratch rather than obtained by post-hoc linearization.

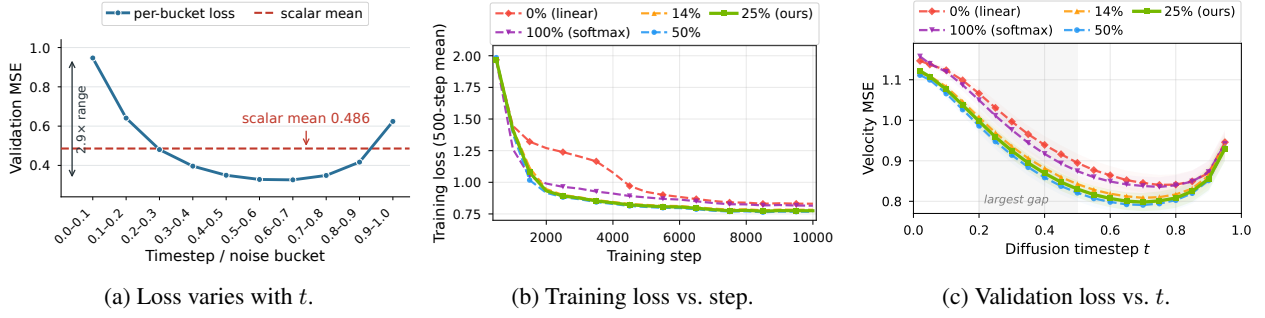


Figure 4 | **Reading loss by timestep, then the softmax-ratio proxy.** (a) On a production checkpoint, validation loss spans a $\sim 3\times$ range across the noise axis, so the scalar random- t mean (dashed) hides structure. We therefore read every proxy comparison per timestep rather than as one scalar. (b,c) Softmax-ratio proxy (5B, 256p/81f): (b) 500-step mean training loss and (c) held-out normalized-latent velocity MSE at 10K (200 videos; pointwise 95% CIs) for five ratios.

Axis 2: Residual attention. With the 3:1 hybrid ratio fixed, we study AttnRes across training and select its router design (Table 3).

Table 3 | **AttnRes training and design probes.** (a) Late-stage continuation; (b–d) early short-run probes. Wins counts improved noise buckets; green shading marks the selected designs.

(a) Router effect			(b) Explicit t input		(c) Query sharing			(d) Block span (S)			
Design	MSE↓	Wins	Setting	Loss↓	Design	Loss↓	Mem	S	Loss↓	Lat.	Mem
No AttnRes	0.48547	—	with t	0.962	per-layer	0.496	+16%	4	0.965	+3.9%	+2.9%
+ AttnRes	0.48506	17/20	without t	0.920	shared	0.495	+4%	8	0.962	+3.1%	+2.1%
								16	0.962	+3.1%	+1.6%

Router across training (Table 3a,b). During early training, short-run probes select the router formulation: removing its explicit timestep input improves loss from 0.962 to 0.920. After the hybrid backbone has matured, a continued-training comparison finds AttnRes and its removal reach comparable held-out MSE (0.4855 vs. 0.4851, with 17 of 20 noise buckets slightly favoring AttnRes). We do not read a quality advantage from this narrow margin; we adopt AttnRes for the cross-depth reuse it provides and use a timestep-independent router, verifying the intended behavior through the rank and routing probes in Figure 6.

Query design (Table 3c). Per-layer and shared projections have nearly identical loss (0.496 vs. 0.495), but sharing reduces memory overhead $4\times$ (+4% vs. +16%). Despite sharing queries across depth, the selected router stays feature-dependent: across five denoising timesteps its mean normalized routing-entropy variation is $2.15\times$ that of the per-layer design (Figure 6a; protocol in Appendix E.3). **Block size (Table 3d).** $S \in \{4, 8, 16\}$ yield similar loss point estimates (0.962–0.965), so the short sweep does not identify a preferred span. Memory scales with block count: $S=4$ uses +2.9% while $S=16$ uses only +1.6%. We keep $S=8$ as an engineering default because it gives two regular 3:1 attention cycles per aggregation block, preserves more intermediate depth sources than $S=16$, and keeps overhead near $S=16$.

The selected shared router remains adaptive without an explicit timestep input: its routing entropy varies across denoising stages through the timestep-conditioned source features. Direct perturbation and weight-space analyses provide the complementary explanation. Replacing the learned router offset by its mean or evaluating it at the wrong timestep changes validation loss by at most 0.0002, while the offset has 0.979 mean cross-timestep cosine similarity and a time-varying residual only 13% as large as its constant component (Appendix E.4).

5.4. Efficiency Scaling

Resolution and router scaling. The proxy studies select a shared, timestep-independent router with $S=8$. We compare the 25%-softmax backbone with full softmax under one compiled, no-AttnRes protocol in which every variant uses its best kernels (fused linear attention, FlashAttention for all softmax layers). This is the standard protocol for all DiT-forward comparisons in the paper; compact reference values are in Appendix Table 8.

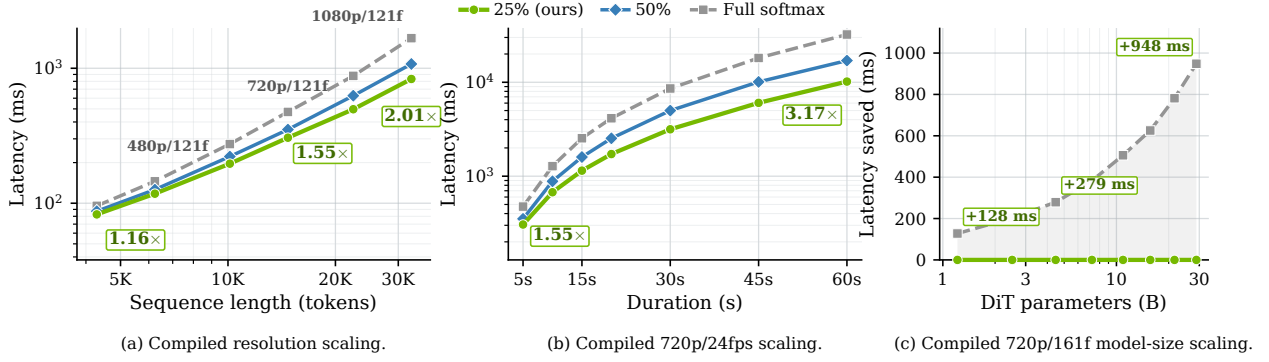


Figure 5 | **Hybrid-attention DiT-forward scaling** (one H100, bf16, batch one; AttnRes off). (a) Compiled resolution scaling. (b) Compiled 720p/24 fps duration scaling; the 121-frame point is shared with (a). (c) Full-softmax latency minus 25% hybrid across model sizes at 720p/161f (19.3K tokens), using the same compiled backend and two run orders. Only (a,b) include 50% softmax; these shape profiles are not quality results.

Sequence- and model-size scaling. Figure 5(a) compares six 480p–1080p shapes and three attention ratios. Under the compiled best-kernels no-AttnRes protocol, full/25%-hybrid speedup rises from 1.16 $\times$ to 2.01 $\times$. At fixed 720p and 24 fps it reaches 3.17 $\times$ at the 60 s tensor shape (1,441 frames, Figure 5(b)). Under the same protocol, panel (c) fixes the 720p/10 s shape and jointly grows width and depth from 1.2B to 28.9B. The hybrid is faster at all eight scales, and its absolute saving grows from 128 to 948 ms. A controlled decomposition shows the two axes act differently: at fixed width, growing depth from 16 to 60 layers leaves the speedup nearly flat (1.41 $\times$), since depth multiplies both variants alike, whereas at fixed depth, growing width shifts it from 1.45 $\times$ to 1.33 $\times$ as the width-bound FFN and projection cost outpaces attention (Appendix F.2). The hybrid advantage is thus governed by how much of the compute is sequence-bound, which is exactly the regime long video occupies.

Model and hardware scaling. Matched H100/GB200 profiling shows that the hybrid scales across hardware generations. At 736 $\times$ 1280 $\times$ 121, one 14B hybrid forward decreases from 768.5 ms on H100 to 389.1 ms on GB200 (1.98 $\times$). Across the full shape sweep through 125.1K latent tokens, GB200 provides a 1.66–1.98 $\times$ gain (median 1.84 $\times$; Appendix F.3). The architecture advantage also persists on both devices: replacing the 12-anchor hybrid with full softmax increases Full/Hybrid speedup with 720p duration from 1.47 $\times$ to 2.81 $\times$ on H100 and from 1.60 $\times$ to 3.10 $\times$ on GB200 (Appendix F.4). These are matched systems profiles rather than checkpoint-quality results.

Kernel-friendliness and deployment. The backbone stays close to standard attention and FFN primitives: fixed softmax anchors and a conv-free SwiGLU FFN, with no convolution-adapter path. Removing SANA-Video’s temporal-convolution FFN avoids a measured 20–29% overhead at the larger scale (Appendix F.6) and leaves the attention ratio as the main sequence-scaling variable. Every DiT-forward comparison above uses equally optimized compiled implementations (FlashAttention for softmax and fused linear attention). At long sequences, the remaining softmax anchors dominate module time (Appendix Figure 18), motivating the separate deployment stack below.

Full stack inference acceleration with Sol-Engine. We optimize SANA-Video 2.0 through a three-stage Sol-Engine deployment stack [18] on NVIDIA B200 at 736 $\times$ 1280 $\times$ 193 (720p, 8 s). Kernel and execution optimization first reduces end-to-end latency from 62.65 to 30.74 s. Residual reuse then evaluates 33 of 50 denoising steps and reaches 20.89 s, after which sparse attention on the later softmax anchors reaches 17.52 s, for a directly measured 3.58 $\times$ speedup. These are orthogonal deployment improvements rather than modeling contributions: fusion preserves the computation, while caching and sparse attention introduce approximation. Appendix F.10 gives the implementation and stage-wise protocol.

5.5. Mechanistic Analysis

State-rank recovery. The ratio sweep shows that periodic softmax anchors improve the proxy quality–efficiency tradeoff. We next test whether AttnRes realizes the complementary goal of carrying richer representations across

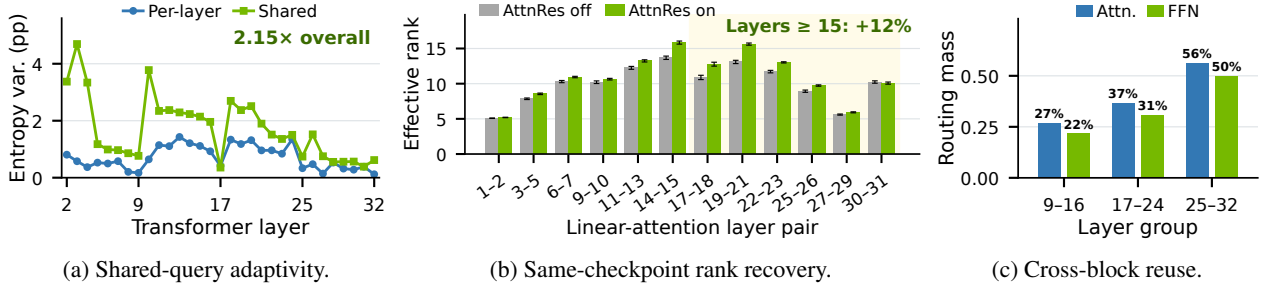


Figure 6 | **Evidence for the selected AttnRes design.** (a) A shared query remains feature-adaptive across timesteps. (b) Enabling depth aggregation recovers effective rank in deeper linear layers. (c) Learned routing increasingly reuses completed blocks.

depth. Using effective rank, the exponential entropy of the linear-state singular spectrum, we toggle depth aggregation on and off on the same production checkpoint under matched prompt and noise seeds. At maximum noise, enabling AttnRes increases the mean effective rank of the deeper layers by 11.7%, with non-decreasing rank in 22 of 24 linear layers (Figure 6b). The static router adds a negligible parameter cost (well below 0.001% of the model).

Cross-depth reuse. On the same architecture-family checkpoint, the router does not collapse to the input or current partial sum: completed-block mass rises with depth and reaches 56%/50% for attention/FFN in the deepest blocks (Figure 6c). This reuse is structured rather than diffuse. Within the completed blocks the router favors the most recently finished one, allocating 26.0%/25.3% of attention/FFN mass to the nearest completed block versus 15.5%/13.2% and 14.5%/11.1% to the two earlier blocks, a recency gradient that holds across depth. Removing completed sources at block entries reduces effective rank by 82–91%, while the same removal at mid-block layers barely changes it (Appendix E.6), localizing the reused information to block boundaries where the partial sum has just reset. Together with the rank recovery above, this evidences cross-depth reuse at the representation level rather than only in the routing weights.

6. Applications

Physical AI [junsong: for yuyang]

7. Related Work

Most open video generators build on diffusion transformers with quadratic softmax token mixing, including Wan 2.1/2.2, HunyuanVideo, CogVideoX, and Open-Sora [14, 25, 34, 40], with MAGI-1 [28] exploring autoregressive generation and LTX-Video [8] attacking inference cost through aggressive latent compression. Efficient alternatives replace the $N \times N$ attention map with a fixed-size state via linear attention [1, 10, 39] or state-space models [7, 16, 32], at the cost of a rank bottleneck [4]. SANA-Video [3] demonstrates a pure-linear video DiT with a temporal-convolution FFN, and the SANA family further extends efficient backbones to world modeling and streaming editing [45, 47]. Hybrid designs keep a minority of softmax layers: Qwen3-Next [27] and Kimi-Linear [11] interleave gated linear and softmax attention at a 3:1 ratio, and the recent Kimi K3 [12] pairs this hybrid-linear attention with Attention Residuals [13] at trillion-parameter scale. Post-SDPA output gating is studied separately [26], and in vision DiG [48] applies gated linear attention to image DiTs while Attention Surgery [6] linearizes pretrained video DiTs. A complementary line keeps the softmax layers but sparsifies them: Sparse-VideoGen [36], Radial Attention [17], SpargeAttn [44], PISA [15], and Native Sparse Attention [42] reduce the cost of each remaining operator rather than how many layers invoke it. SANA-Video 2.0 brings this hybrid-linear-attention-plus-AttnRes recipe, so far demonstrated only in language models, to video generation, trained entirely from scratch. Rather than transplanting the language-model configuration wholesale, we re-ablate it for the video regime: we sweep the softmax ratio (confirming a 25% anchor knee), aggregate Block AttnRes over completed depth summaries with a depth-shared rather than per-layer query, and, because diffusion adds a denoising timestep absent from autoregressive LLMs, drop the explicit router timestep conditioning that our ablations find unnecessary (Section 5.3). We further apply sparse anchor kernels only to the eight fixed softmax layers. Appendix G gives the broader comparison.

8. Conclusion

We presented SANA-Video 2.0, a scratch-initialized video DiT that combines 25% softmax anchors with depth-shared Block AttnRes, with short proxies selecting 25% as the practical quality–efficiency knee. With 40-step sampling, the model reaches VBench Total 84.30 in 13.2 s at $480 \times 832 \times 81$ on one H100. AttnRes raises deep-layer effective rank by $\sim 12\%$, and the DiT forward runs $3.2\times$ faster than full softmax at a 720p/60 s shape under matched best-kernel compilation. Remaining limitations are the short proxy studies and the shape-only long-duration profiling. Causal or streaming extensions would require a recurrent operator.

References

- [1] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. Simple Linear Attention Language Models Balance the Recall-Throughput Tradeoff. In *ICML*, 2024.
- [2] Shuo Cao, Jiayang Li, Xiaohui Li, Yuandong Pu, Kaiwen Zhu, Yuanting Gao, Siqi Luo, Yi Xin, Qi Qin, Yu Zhou, Xiangyu Chen, Wenlong Zhang, Bin Fu, Yu Qiao, and Yihao Liu. UniPercept: Towards Unified Perceptual-Level Image Understanding across Aesthetics, Quality, Structure, and Texture. In *ICML*, 2026.
- [3] Junsong Chen, Yuyang Zhao, Jincheng Yu, Ruihang Chu, Junyu Chen, Shuai Yang, Xianbang Wang, Yicheng Pan, Daquan Zhou, Huan Ling, Haozhe Liu, Hongwei Yi, Hao Zhang, Muiyang Li, Yukang Chen, Han Cai, Sanja Fidler, Ping Luo, Song Han, and Enze Xie. SANA-Video: Efficient Video Generation with Block Linear Diffusion Transformer. arXiv preprint arXiv:2509.24695, 2025.
- [4] Qihang Fan, Huaibo Huang, and Ran He. Breaking the Low-Rank Dilemma of Linear Attention. In *CVPR*, 2025.
- [5] Gemma Team. Gemma 2: Improving Open Language Models at a Practical Size. arXiv preprint arXiv:2408.00118, 2024.
- [6] Mohsen Ghafoorian, Denis Korzhnikov, and Amirhossein Habibian. Attention Surgery: An Efficient Recipe to Linearize Your Video Diffusion Transformer. arXiv preprint arXiv:2509.24899, 2025.
- [7] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. In *Conference on Language Modeling (COLM)*, 2024.
- [8] Yoav HaCohen, Nisan Chiprut, Benny Brazowski, Daniel Shalem, et al. LTX-Video: Realtime Video Latent Diffusion. arXiv preprint arXiv:2501.00103, 2025.
- [9] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive Benchmark Suite for Video Generative Models. In *CVPR*, 2024.
- [10] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. In *ICML*, 2020.
- [11] Kimi Team. Kimi Linear: An Expressive, Efficient Attention Architecture. arXiv preprint arXiv:2510.26692, 2025.
- [12] Kimi Team. Kimi K3: Open Frontier Intelligence. Moonshot AI technical blog, <https://www.kimi.com/blog/kimi-k3>, 2026.
- [13] Kimi Team. Attention Residuals. arXiv preprint arXiv:2603.15031, 2026.
- [14] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, et al. HunyuanVideo: A Systematic Framework For Large Video Generative Models. arXiv preprint arXiv:2412.03603, 2024.
- [15] Haopeng Li, Shitong Shao, Wenliang Zhong, Zikai Zhou, Lichen Bai, Hui Xiong, and Zeke Xie. PISA: Piecewise Sparse Attention Is Wiser for Efficient Diffusion Transformers. arXiv preprint arXiv:2602.01077, 2026.
- [16] Kunchang Li, Xinhao Li, Yi Wang, Yinan He, Yali Wang, Limin Wang, and Yu Qiao. VideoMamba: State Space Model for Efficient Video Understanding. In *ECCV*, 2024.
- [17] Xingyang Li et al. Radial Attention: $O(n \log n)$ Sparse Attention with Energy Decay for Long Video Generation. arXiv preprint arXiv:2506.19852, 2025.
- [18] Yitong Li, Junsong Chen, Haopeng Li, Haozhe Liu, Jincheng Yu, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Sol Video Inference Engine: Agent-Native Full-Stack Acceleration Framework for Efficient Video Generation. arXiv preprint arXiv:2606.23743, 2026.
- [19] Zhi Li, Anne Aaron, Ioannis Katsavounidis, Anush Moorthy, and Megha Manohara. Toward A Practical Perceptual Video Quality Metric. Netflix Technology Blog, <https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652>, 2016.

- [20] Chenchen Liu, Junyi Chen, Lei Li, Lu Chi, Mingzhen Sun, Zhuoying Li, Yi Fu, Ruoyu Guo, Yiheng Wu, Ge Bai, and Zehuan Yuan. Bernini: Latent Semantic Planning for Video Diffusion. arXiv preprint arXiv:2605.22344, 2026.
- [21] Yijun Liu, Jie Huang, Zeyue Xue, Yuming Li, Ruizhe He, Haoran Li, Shijia Ge, and Siming Fu. HPSv3++: Scaling Reward Models Across the Full Spectrum of Diffusion Model Capabilities. arXiv preprint arXiv:2606.14657, 2026.
- [22] Yue Liu, Yunjie Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, Jianbin Jiao, and Yunfan Liu. VMamba: Visual State Space Model. In *NeurIPS*, 2024.
- [23] Xiangyang Luo, Qingyu Li, Yuming Li, Guanbo Huang, Yongjie Zhu, Wenyu Qin, Meng Wang, Pengfei Wan, and Shao-Lun Huang. Beyond the Golden Data: Resolving the Motion-Vision Quality Dilemma via Timestep Selective Training. In *CVPR*, 2026.
- [24] William Peebles and Saining Xie. Scalable Diffusion Models with Transformers. In *ICCV*, 2023.
- [25] Xiangyu Peng, Zangwei Zheng, Chenhui Shen, et al. Open-Sora 2.0: Training a Commercial-Level Video Generation Model in \$200k. arXiv preprint arXiv:2503.09642, 2025.
- [26] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, and Junyang Lin. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. arXiv preprint arXiv:2505.06708, 2025.
- [27] Qwen Team. Qwen3-Next: Towards Ultimate Training & Inference Efficiency. Qwen Team blog, https://qwenlm.github.io/blog/qwen3_next/, 2025.
- [28] Sand.ai, Hansi Teng, Hongyu Jia, Lei Sun, Lingzhi Li, et al. MAGI-1: Autoregressive Video Generation at Scale. arXiv preprint arXiv:2505.13211, 2025.
- [29] Team Seedance et al. Seedance 2.0: Advancing Video Generation for World Complexity. arXiv preprint arXiv:2604.14148, 2026.
- [30] Tomáš Souček and Jakub Lokoč. TransNet V2: An Effective Deep Network Architecture for Fast Shot Transition Detection. arXiv preprint arXiv:2008.04838, 2020.
- [31] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568:127063, 2024.
- [32] Yao Teng, Yue Wu, Han Shi, Xuefei Ning, Guohao Dai, Yu Wang, Zhenguo Li, and Xihui Liu. DiM: Diffusion Mamba for Efficient High-Resolution Image Synthesis. arXiv preprint arXiv:2405.14224, 2024.
- [33] Bram Wallace, Meihua Dang, Rafael Rafailov, Linqi Zhou, Aaron Lou, Senthil Purushwalkam, Stefano Ermon, Caiming Xiong, Shafiq Joty, and Nikhil Naik. Diffusion Model Alignment Using Direct Preference Optimization. In *CVPR*, 2024.
- [34] Wan Team, Ang Wang, Baole Ai, Bin Wen, et al. Wan: Open and Advanced Large-Scale Video Generative Models. arXiv preprint arXiv:2503.20314, 2025.
- [35] Haoning Wu, Erli Zhang, Liang Liao, Chaofeng Chen, Jingwen Hou, Annan Wang, Wenxiu Sun, Qiong Yan, and Weisi Lin. Exploring Video Quality Assessment on User Generated Contents from Aesthetic and Technical Perspectives. In *ICCV*, 2023.
- [36] Haocheng Xi et al. Sparse VideoGen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity. In *ICML*, 2025. arXiv:2502.01776.
- [37] Haofei Xu, Jing Zhang, Jianfei Cai, Hamid Rezaatofghi, Fisher Yu, Dacheng Tao, and Andreas Geiger. Unifying Flow, Stereo and Depth Estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11):13941–13958, 2023.
- [38] Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and Evaluating Human Preferences for Text-to-Image Generation. In *NeurIPS*, 2023.

- [39] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated Linear Attention Transformers with Hardware-Efficient Training. In *ICML*, 2024.
- [40] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, et al. CogVideoX: Text-to-Video Diffusion Models with An Expert Transformer. In *ICLR*, 2025.
- [41] Zhiyuan You, Xin Cai, Jinjin Gu, Tianfan Xue, and Chao Dong. Teaching Large Language Models to Regress Accurate Image Quality Scores Using Score Distribution. In *CVPR*, 2025.
- [42] Jingyang Yuan et al. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. arXiv preprint arXiv:2502.11089, 2025.
- [43] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid Loss for Language Image Pre-Training. In *ICCV*, 2023.
- [44] Jintao Zhang et al. SpargeAttn: Accurate Sparse Attention Accelerating Any Model Inference. In *ICML*, 2025. arXiv:2502.18137.
- [45] Yuyang Zhao, Yicheng Pan, Qiyuan He, Jincheng Yu, Junsong Chen, Tian Ye, Haozhe Liu, Enze Xie, and Song Han. SANA-Streaming: Real-time Streaming Video Editing with Hybrid Diffusion Transformer. arXiv preprint arXiv:2605.30409, 2026.
- [46] Xin Zhou, Dingkan Liang, Kaijin Chen, Tianrui Feng, Xiwu Chen, Hongkai Lin, Yikang Ding, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Less is Enough: Training-Free Video Diffusion Acceleration via Runtime-Adaptive Caching. arXiv preprint arXiv:2507.02860, 2025.
- [47] Haoyi Zhu, Haozhe Liu, Yuyang Zhao, Tian Ye, Junsong Chen, Jincheng Yu, Tong He, Song Han, and Enze Xie. SANA-WM: Efficient Minute-Scale World Modeling with Hybrid Linear Diffusion Transformer. arXiv preprint arXiv:2605.15178, 2026.
- [48] Lianghui Zhu, Zilong Huang, Bencheng Liao, Jun Hao Liew, Hanshu Yan, Jiashi Feng, and Xinggang Wang. DiG: Scalable and Efficient Diffusion Models with Gated Linear Attention. In *CVPR*, 2025.

A. Model and Training Details

A.1. Model Configuration

SANA-Video 2.0 has 5B and 14B configurations that share the selected hybrid design. Table 4 records their complete architecture and training hyperparameters.

Table 4 | **Full 5B and 14B configurations of SANA-Video 2.0.** Both configurations use the same hybrid-attention design.

Setting	5B	14B
<i>Backbone architecture</i>		
Depth	32	48
Hidden dimension	2,560	3,584
Linear attention	20 heads ($d_h=128$)	28 heads ($d_h=128$)
Softmax attention	10 heads ($d_h=256$)	14 heads ($d_h=256$)
Attention operators	Gated bidirectional linear / gated softmax; self- and cross-attention QK normalization	
Softmax anchors	8 layers; 25% (3:1)	12 layers; 25% (3:1)
AttnRes	$S=8$, shared projection, no time conditioning	$S=8$, shared projection, no time conditioning
FFN	SwiGLU, ratio 4.0	SwiGLU, ratio 4.0
Position encoding	Wan RoPE; separate linear/softmax head dimensions	
Patch size	(1, 1, 1)	(1, 1, 1)
Parameters	$\sim 4.5\text{B}$ (5B class)	$\sim 13.1\text{B}$ (14B class)
<i>Training configuration</i>		
Resolution / frames	480p–720p; 81/121/193	480p–720p; 81/121/193
VAE	LTX-VAE 2.3, 128 channels	LTX-VAE 2.3, 128 channels, causal encode
VAE stride	$8 \times 32 \times 32$ (temporal $\times$ spatial $\times$ spatial)	
Text encoder	Gemma-2-2B-IT, 300 tokens	
Objective	Flow matching; stage-specific TQD / Self-Flow	Flow matching + Self-Flow
Self-Flow	student/teacher 9/25, weight 0.8, $R_M=0.1$	student/teacher 14/34, weight 0.8, $R_M=0.1$
Noise distribution	shift 1 then token-aware 3–6, $\sigma=0.95$	tail-floored logit-normal, shift 3, $\sigma=0.95$
Optimizer	AdamW, lr 10^{-4} / 5×10^{-5}	AdamW, lr 10^{-4} , $\beta=(0.9, 0.999)$, wd 0
Schedule / clipping	constant + 500-step warmup; clip 0.1	constant + 500-step warmup; clip 0.1
Per-rank batch	1 video	1 video / 4 images; no accumulation
Precision / sharding	bf16, FSDP	bf16, FSDP + activation checkpointing
Training hardware	$64 \times \text{H100}$	$192 \times \text{GB200}$ (Poly)

A.2. Self-Flow Distillation and Dual-Timestep Tokens

Self-Flow is an auxiliary training aid applied only during pre-training and continual training (disabled for SFT). It adds a feature-distillation loss from a shallow student readout toward a deeper (or, in later continual training, EMA) teacher, and a dual-timestep schedule that assigns a small token partition ($R_M=0.1$) to a second timestep. All tokens stay in the flow-matching loss, so it changes conditioning rather than reducing backbone compute; the exact layer indices and weights are in Table 4.

A.3. Content-aware Flow-Shift (TQD)

TQD [23] uses curation scores to place supervision where each pre-training clip is most informative. With base flow-shift 1, high-motion clips (fps-normalized UniMatch score above 30) receive a +1.1 logit bias toward high noise, high-quality clips (DOVER above 0.91) receive a -1.1 bias toward low noise, and the remainder stay centered. These biases move the median timestep to approximately 0.75/0.25, equivalent to an effective shift near 3 (Figure 7). The later token-aware schedule already begins at shift 3, so TQD can be annealed out without an abrupt change in noise coverage.

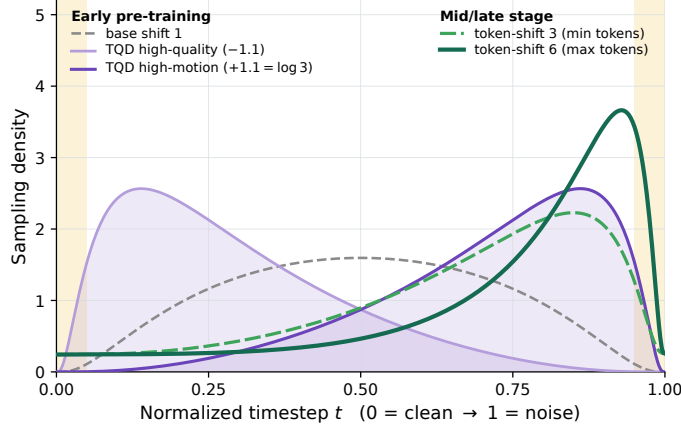


Figure 7 | **Flow-shift handoff across stages.** Pre-training TQD biases high-quality clips toward low noise and high-motion clips toward high noise. The later token-count shift spans 3–6; its minimum matches TQD’s high-noise edge, making the transition continuous.

A.4. Timestep-Stratified Checkpoint Monitoring

A random-timestep mean can hide improvements in one noise regime behind regressions in another. We therefore divide the 1,000-step noise axis into ten equal buckets and compare four checkpoints from one continuous trajectory. Every checkpoint reuses the same 100 clips, text features, noise tensors, and timestep assignments, covering each integer timestep exactly once. The equal-bucket average is a monitoring statistic, not a replacement for the training objective.

From the first to last checkpoint, macro MSE falls by 6.4%, but the bucket view reveals the useful detail: low-noise MSE improves by as much as 11.4%, while the highest-noise bucket regresses by 1.2% (Figure 8). Stratification also reduces the standard deviation of the change estimate by $2.3\times$ relative to IID timestep sampling. VBench Total rises from 82.68 to 83.29 over the same checkpoints; with four observations, this is supporting context rather than a correlation claim.

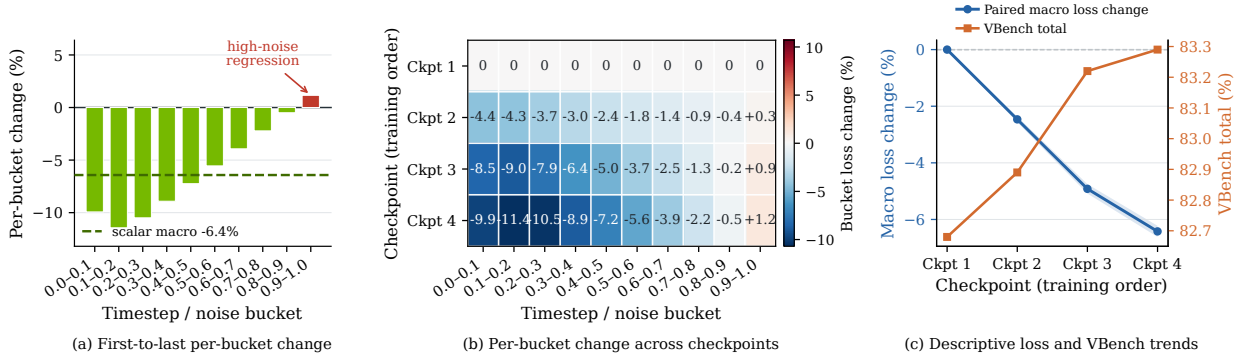


Figure 8 | **Timestep-stratified checkpoint monitoring.** Four matched checkpoints from one run. (a,b) Bucketwise loss changes expose improvements hidden by the scalar mean. (c) Paired macro change with 95% intervals and the descriptive VBench trend.

B. Data Curation and Progressive Selection

We process the corpus with a uniform six-stage pipeline. Its purpose is not only to remove defective clips, but to construct the progressive supervision used by the training curriculum: pre-training favors coverage, continual training raises the quality and motion-consistency floors, and SFT uses the most trusted subset. The same processing logic is used across these pools, with stage-dependent selection thresholds.

Table 5 | **Signals used by the in-house quality/motion funnel.** Selection thresholds tighten progressively across the pre-training, continual, and SFT stages.

Signal family	Examples	Role
Visual quality	DOVER, blur/exposure statistics	Fidelity and aesthetics
Motion	optical flow, VMAF motion	Richness and consistency
Color	saturation and luminance statistics	Natural appearance
Consistency/alignment	image–video, vision–language similarity	Conditioning reliability
Shot integrity	cut and artifact detectors	Temporal continuity

Decode, segment, and clean. We first reject undecodable or undersized clips, normalize metadata, and split long videos into coherent shots with TransNetV2 [30]. Temporally stable crops remove letterboxing and subtitles when possible; clips dominated by overlays, unstable cuts, severe blur, broken exposure, or temporal artifacts are discarded.

Score complementary properties. The remaining clips are evaluated along separate quality, motion, color, and consistency axes rather than collapsed into one score (Table 5). DOVER measures visual quality, UniMatch and VMAF-derived features describe motion, and SigLIP features measure visual–text consistency [19, 35, 37, 43]. Decisions are aggregated over time, and optical flow is normalized by frame rate so informative low-motion clips are not removed simply for being slow.

Build the curriculum. Progressively stricter per-axis thresholds form a broad pre-training pool, a cleaner continual-training pool, and a small trusted SFT subset. Captions follow the same progression, ending in a structured description of subjects, actions, camera motion, scene, lighting, interactions, and temporal evolution. Quantized motion descriptors are appended when available so motion remains explicit to the text conditioner.

C. Evaluation and Measurement Protocols

C.1. Sampling and VBench Evaluation

The reported SANA-Video 2.0 operating points use 40 flow-DPM-Solver steps, motion bucket 30, seed 0, and Gemma-2-2B-IT text features [5]. The 81-frame, 16-fps setting uses classifier-free guidance 6.0 and flow-shift 6.0; the 121/193-frame, 24-fps settings use guidance 8.0 and flow-shift 12.0. The 81-frame result comes from one late-stage production checkpoint, while the 121/193-frame results share a second late-stage checkpoint. Baselines retain their model-specific inference settings.

We evaluate the full VBench [9] text-to-video suite with the official prompts and standard 16 dimensions. Quality aggregates seven dimensions and Semantic aggregates nine; Total uses VBench’s default 4:1 quality–semantic weighting. Table 6 abbreviates the dimensions as SC/BC (subject/background consistency), TF (temporal flickering), MS (motion smoothness), DD (dynamic degree), AQ/IQ (aesthetic/imaging quality), OC (object class), MO (multiple objects), HA (human action), CO (color), SR (spatial relationship), SE (scene), AS/TS (appearance/temporal style), and OV (overall consistency).

Score sources. All baseline scores are either official results (★: official 720p leaderboard, ‡: author-reported) or measured by us with each model’s official pipeline and default inference settings (†). Unmarked scores use the common 480×832×81 protocol. Table 2 shows the compact selection with the 81-frame SANA-Video 2.0 result, and Table 6 adds the full baseline set and our 121-/193-frame operating points. Wan 2.2 is A14B active (27B total).

Table 6 | **Complete VBench comparison across all 16 dimensions (%)**. Green intensity ranks the top three results per column (best in bold); dashes mark models with only aggregate scores reported. Appendix C defines the score-source markers and protocols.

Model	Aggregates			Quality dimensions							Semantic dimensions								
	Total↑	Quality↑	Semantic↑	SC	BC	TF	MS	DD	AQ	IQ	OC	MO	HA	CO	SR	SE	AS	TS	OV
LTX-2.3 (base)†	81.95	83.85	74.35	93.10	95.77	98.92	98.71	71.39	62.13	67.51	88.89	60.26	95.80	81.31	68.59	51.38	21.11	24.31	25.91
CogVideoX 1.5	82.17	82.78	79.76	96.87	97.35	98.88	98.31	50.93	62.79	65.02	87.47	69.65	97.20	87.55	80.25	52.91	24.89	25.19	27.30
Cosmos-3 Nano†	83.13	84.29	78.50	95.05	97.26	99.08	99.06	60.28	62.71	69.37	89.29	78.03	96.60	89.20	72.72	54.56	20.98	24.49	26.78
Lance†	83.18	84.09	79.55	94.68	94.95	98.34	98.92	71.11	63.71	67.71	94.18	76.69	97.60	81.33	79.56	52.83	23.12	24.58	26.89
Wan 2.1 (1.3B)	83.31	85.23	75.65	97.56	97.93	99.55	98.52	65.19	65.46	67.01	88.81	74.83	94.00	89.20	73.04	41.96	21.81	23.13	25.50
HunyuanVideo*	83.43	85.07	76.88	97.22	97.60	99.39	99.05	71.94	60.28	67.24	83.48	66.71	94.40	89.79	72.13	54.46	22.21	24.52	26.95
Wan 2.1 (14B)	83.69	85.59	76.11	97.52	98.09	99.46	98.30	65.46	66.07	69.43	86.28	69.58	95.40	88.59	75.39	45.75	22.64	23.19	25.91
SANA-Video	84.17	84.85	81.46	97.13	97.71	98.37	98.20	69.17	68.27	64.82	95.39	85.05	95.40	89.43	78.00	57.83	22.43	24.31	27.04
Wan 2.2 (A14B)*	84.23	85.42	79.50	97.29	97.39	99.22	98.16	61.02	67.22	71.75	94.06	82.10	96.40	87.43	78.39	56.80	20.39	23.64	26.12
Open-Sora 2.0	84.34	85.40	80.12	97.71	98.00	99.40	98.69	71.39	64.39	65.66	94.50	77.72	95.40	85.98	76.18	52.71	22.98	25.91	27.50
Bernini-R†	84.64	85.18	82.49	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
SANA-Video 2.0 (81f)	84.30	85.61	79.05	97.34	97.33	97.84	98.31	78.89	67.99	66.51	92.80	76.75	94.40	88.05	71.85	59.88	21.35	24.05	26.93
SANA-Video 2.0 (121f)	85.29	86.98	78.51	97.44	97.09	97.58	98.40	92.50	68.85	68.39	92.39	74.38	95.00	89.17	69.34	58.81	21.26	24.46	26.64
SANA-Video 2.0 (193f)	84.48	86.51	76.37	98.00	96.94	97.94	98.52	88.33	68.10	66.31	91.76	72.39	92.60	88.21	65.32	54.68	21.27	23.48	26.08

Latency protocol. End-to-end latency covers text encoding, denoising, and VAE decoding at batch size one, excluding checkpoint loading and video writing. Unless noted otherwise, we report the mean of three steady-state videos after one full warmup on a single H100. Baselines use their official pipelines and native 720p-class shapes; the headline 14B comparison standardizes Wan 2.2-A14B, Bernini-R, and both SANA-Video 2.0 scales to 40 steps.

At $736 \times 1280 \times 81$, the 5B and 14B configurations take 30.9 and 69.3 s. At the matched $480 \times 832 \times 81$ shape they take 13.2 and 29.1 s, versus 421.3 s for Bernini-R. The 14B timing executes the complete architecture, including text conditioning and decoding, but uses random weights because no trained 14B checkpoint is available; it is therefore a systems result, not a quality result. For context, SANA-Video-2B officially reports 36 s at 720p [3].

C.2. Latency and Profiling Protocols

The standard architecture profiles use one H100 80GB, bf16, batch size one, eight warmups, and 20 CUDA-event timed forwards (median). Every attention ratio receives its best implementation: fused linear attention, FlashAttention for softmax, and `torch.compile`. Figures 5(a,b) vary resolution and duration with AttnRes disabled; panel (c) follows the same compiled protocol while scaling matched random-initialized backbones at a fixed 19.3K-token shape. These are DiT-forward measurements and exclude text encoding and VAE decoding.

The module-composition figure is a separate eager diagnostic with the selected shared router enabled. It attributes non-overlapping CUDA-event intervals to top-level modules at 22.1K and 111.3K tokens; it is used only to identify bottlenecks, not as a second speedup protocol or as evidence about learned routing. Results from different protocols are never multiplied unless a table explicitly defines the composition.

14B cross-hardware protocol. The H100 and GB200 sweeps use identical software, random inputs, and compiled kernels. The hybrid has 36 fused-linear layers and 12 Flash-SDPA anchors; the full-softmax control sends all 48 layers through the same Flash-SDPA implementation. Width, depth, inputs, and all non-attention settings stay fixed, and each shape runs in an isolated process. Repeated measurements agree within 0.8% on both devices.

D. Qualitative Results

D.1. Additional Qualitative Samples

Figure 9 shows six additional 720p text-to-video clips (1280×736 , 193 frames, 8 s at 24 fps) spanning people, wildlife, urban scenes, and stylized content. Each row unrolls one clip as five frames at 0/2/4/6/8 s, generated with the same 40-step 193-frame sampling recipe as above; the generating prompt is reproduced beneath each row. Four further showcase prompts from the same batches (a violinist in light rain, an octopus changing color, a drone light show, and a desert lightning storm) are held out for the cross-method comparison below (Figure 10).

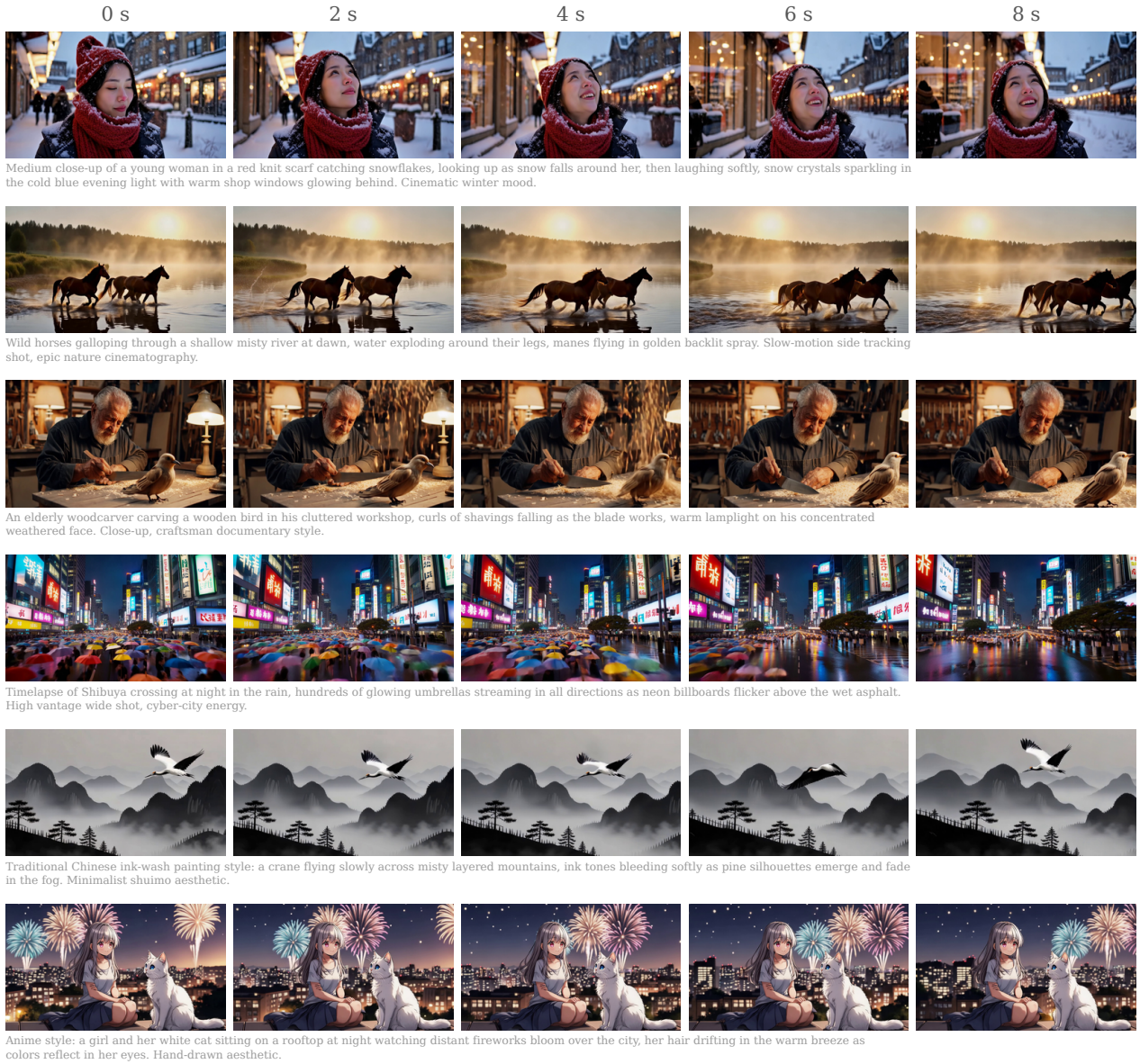


Figure 9 | Additional 720p text-to-video samples. Each row unrolls one 8 s 1280×736 clip as five frames at 0/2/4/6/8 s; the generating prompt is shown beneath each row.

D.2. Qualitative Comparison with Other Methods

Figure 10 compares SANA-Video 2.0 with Wan 2.2-A14B, Bernini-R 14B, Cosmos-3 Nano, and LTX-2.3 on four prompts chosen to expose temporal behavior: sustained action, color change, shape transformation, and repeated events. Each method uses its official pipeline, native 720p-class shape, and protocol step count on one H100 with seed 42; SANA-Video 2.0 uses the 40-step, 193-frame recipe. Because clip durations differ, the figure shows the start, midpoint, and endpoint of each output rather than matching absolute timestamps. The panels are qualitative examples, not an aggregate ranking.

Figures 11 and 12 extend the comparison to image-conditioned generation. Every method receives the same first frame and caption from VBench-I2V, and each row shows normalized progress at 0/25/50/100%. SANA-Video 2.0 generates 121 frames with its image-conditioned checkpoint; LTX-2.3, Wan 2.2 TI2V-5B, and Cosmos-3 use their official first-frame conditioning paths. Bernini-R is omitted because its public pipeline has no corresponding mode. All methods run at native 720p-class shapes with seed 42 on one H100. The examples illustrate differences in motion and composition retention, but do not substitute for a quantitative TI2V benchmark.

E. AttnRes Routing and Representation Analysis

This section explains what the selected router learns and why its final form shares one query across depth without an explicit timestep input. Unless noted otherwise, the analyses use 5B architecture-family checkpoints; the continued-training comparison below uses the mature 4.5B production ancestor.

E.1. Late-Stage Continued-Training Probe

To test the router after useful video representations have formed, we continue a block-level AttnRes run from the same mature 4.5B hybrid ancestor as the no-router trajectory. Near-matched checkpoints are evaluated on the same 100 held-out clips and 20 noise levels. Their mean MSE is effectively tied (0.4851 with AttnRes versus 0.4855 without it), although 17 of 20 buckets slightly favor AttnRes. We use this probe only to rule out degradation; the evidence for cross-depth reuse comes from the routing and representation interventions below, not from this narrow loss difference.

E.2. Routing Selectivity Metric

For each layer l , AttnRes computes aggregation weights $\alpha_l = \text{softmax}(q_l \cdot K)$ over N_l depth sources (initial embedding b_0 , completed block summaries $b_1, b_2, \dots$, and the current partial sum p_l). We measure selectivity via normalized Shannon entropy:

$$\hat{H}_l = \frac{H(\alpha_l)}{\log N_l} = \frac{-\sum_{j=1}^{N_l} \alpha_{l,j} \log \alpha_{l,j}}{\log N_l} \quad (3)$$

For layer l , the attention branch has $N_l^{\text{attn}} = \lceil l/S \rceil + \mathbf{1}[(l-1) \bmod S > 0]$ sources, while the FFN branch has $N_l^{\text{ffn}} = \lceil l/S \rceil + 1$. We omit the first attention routing site ($N = 1$) from normalized-entropy summaries. The normalization otherwise makes source sets of different sizes comparable: $\hat{H}_l = 1$ is uniform and $\hat{H}_l \rightarrow 0$ is selective.

E.3. Timestep Adaptivity

To test whether AttnRes changes its depth aggregation across denoising stages, we evaluate ten held-out batches at five timesteps with real text conditioning. After averaging batches at each timestep, we compute the variation of each routing site s :

$$\text{Adaptivity}_s = \sigma_t[\hat{H}_s]. \quad (4)$$

Higher values indicate greater change across denoising stages. Despite using one query projection across depth, the selected shared router shows $2.15\times$ more entropy variation than the per-layer alternative (Figure 6a). This variation comes from timestep-conditioned source features in the ordinary DiT pathway; it does not require a separate router-timestep offset.

E.4. Timestep Conditioning Probe

To determine whether the explicit router timestep projection learns timestep-specific information or merely a constant bias, we combine a behavioural probe with a direct weight-space analysis of its learned offset $\phi_\tau(t)$.

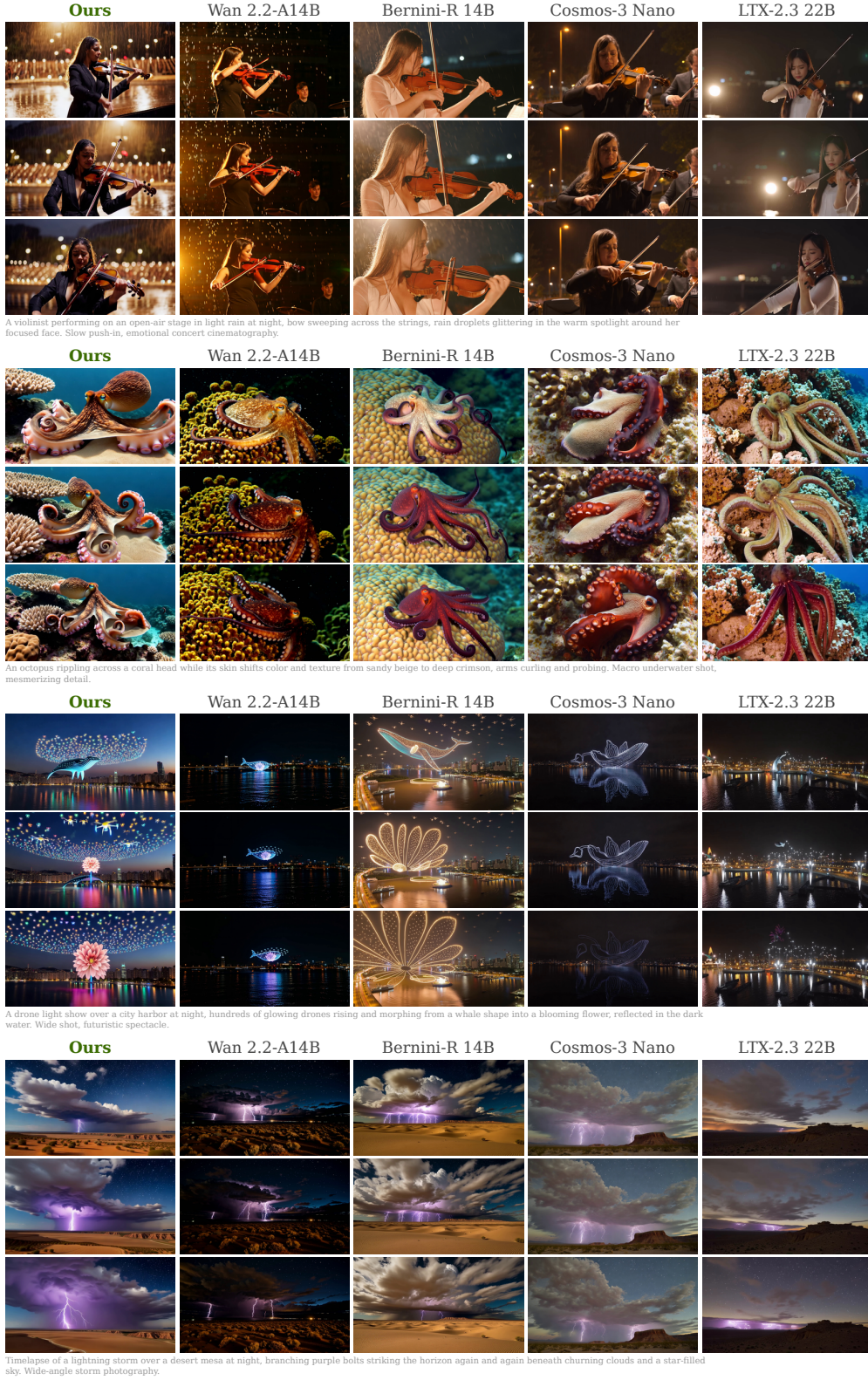


Figure 10 | **Cross-method 720p text-to-video comparison** on four held-out prompts. Columns are methods and rows show 0/50/100% of each clip; prompts appear below the corresponding blocks.

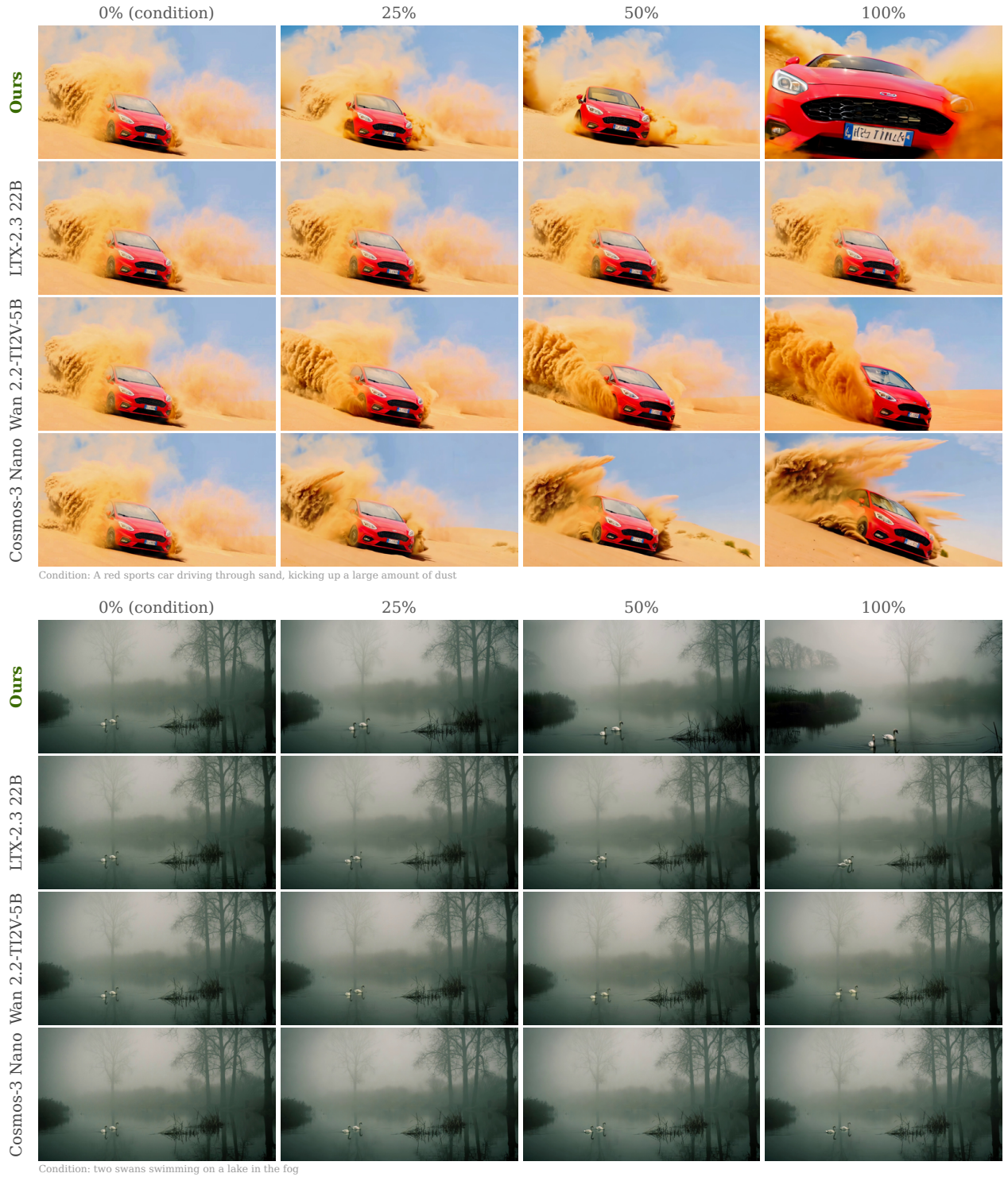


Figure 11 | **First-frame-conditioned video comparison, pairs 1–2.** Each row shows one method at 0/25/50/100% of its clip; the first frame is shared. Protocol details are in Appendix D.2.

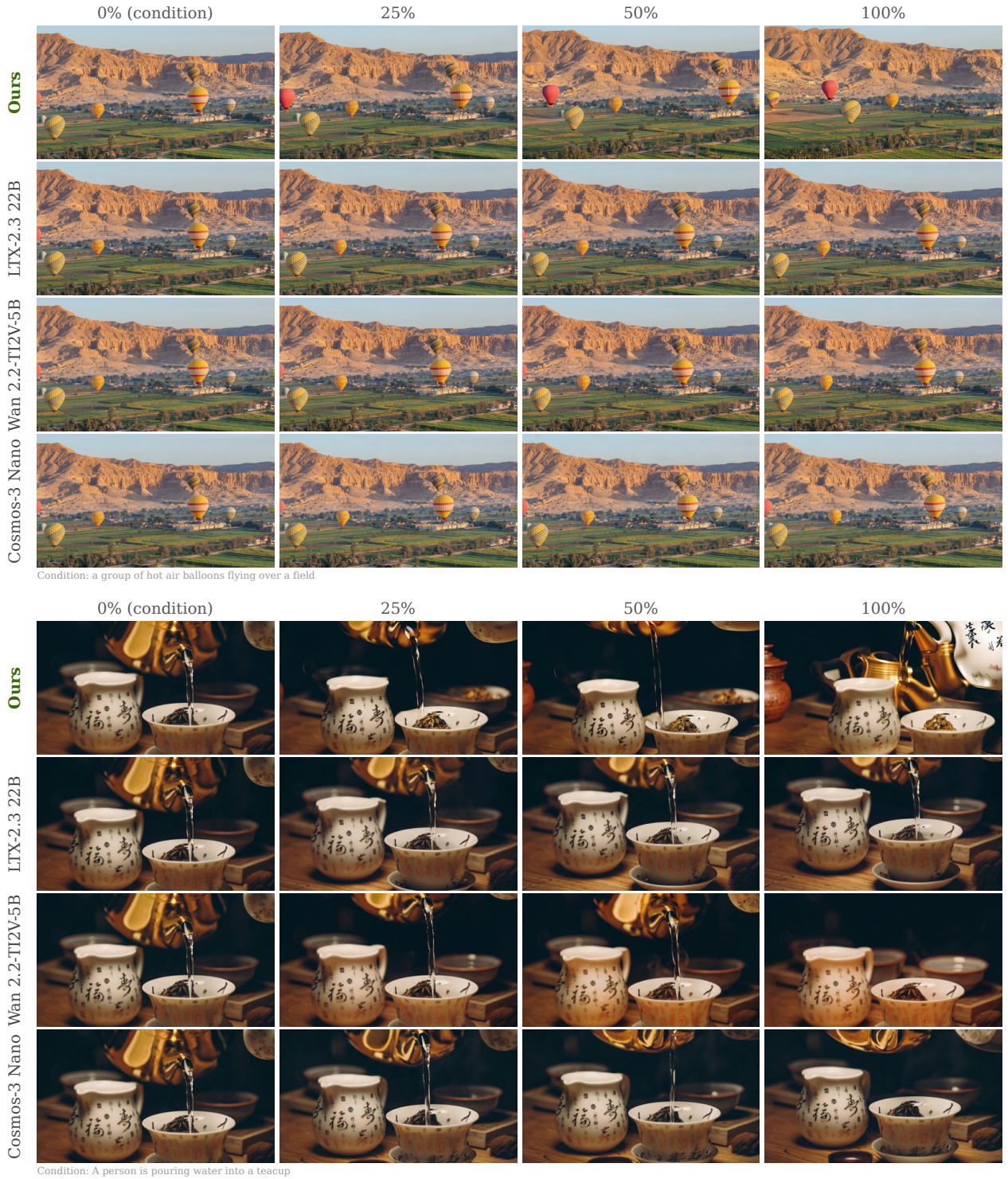
Figure 12 | **First-frame-conditioned video comparison, pairs 3–4.** The protocol matches Figure 11.

Table 7 | **Explicit router-timestep perturbation.** Normal uses the learned offset at the correct timestep; Mean replaces it by its cross-timestep mean; Shuffle evaluates the offset at $1 - t$ while retaining the correct AdaLN timestep.

Router offset	Validation loss↓
Normal	0.4905
Mean	0.4906
Shuffle	0.4907

Behavioural probe. Let q_0 be the base routing query. We evaluate the ablation checkpoint under three conditions (Table 7): **Normal**, $q = q_0 + \phi_\tau(t)$; **Mean**, $q = q_0 + \bar{\phi}_\tau$, where $\bar{\phi}_\tau$ averages 21 uniformly spaced timesteps; and **Shuffle**, $q = q_0 + \phi_\tau(1 - t)$ for AttnRes while AdaLN still receives the correct t . Their aggregate validation-loss point estimates are 0.4905/0.4906/0.4907, a maximum difference of 0.0002 in this probe.

Weight-space probe (Figure 13). Across 21 timesteps, the learned offset remains nearly constant (mean cosine 0.979), and its varying component is only 13% of the constant component’s norm. Together with the negligible behavioural change above, this motivates removing the explicit router-timestep projection while retaining timestep modulation inside every DiT block.

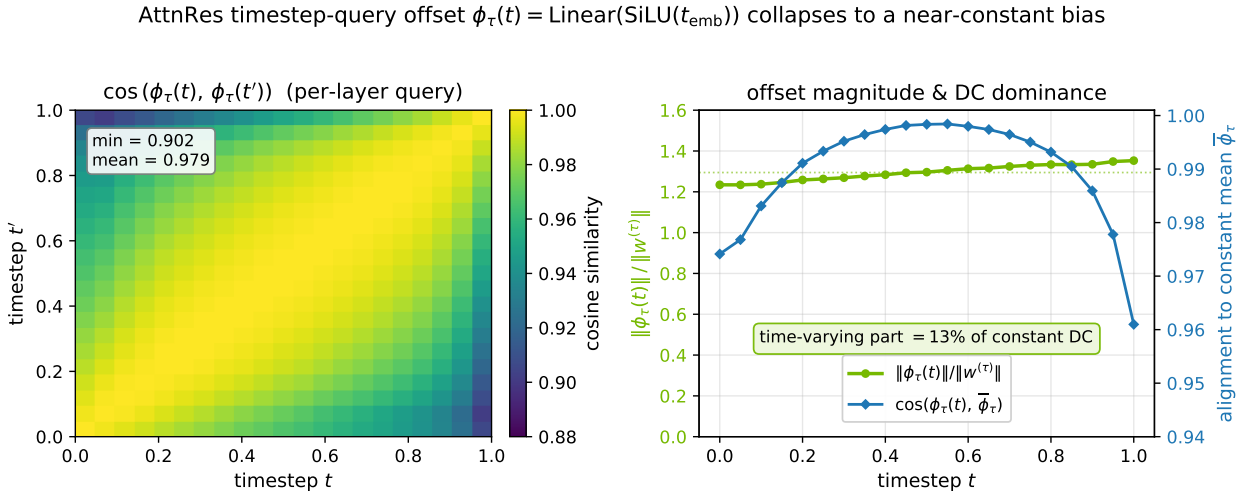


Figure 13 | **The explicit router offset is nearly timestep-invariant.** Across 21 timesteps, $\phi_\tau(t)$ remains strongly aligned with its mean and its time-varying component is small.

E.5. Depth-Routing Patterns

Figures 6c and 14 average routing over ten validation clips. Completed summaries receive roughly half of the routing mass in layers 25–32 (56% for attention and 50% for FFN), and the detailed view favors the most recently completed block. The difference at block entries follows operation order: attention routing occurs before the new partial sum exists, whereas FFN routing occurs afterward. Because each summary contains attention, cross-attention, and FFN updates, this evidence supports cross-depth feature reuse without assigning it to a single operator.

E.6. Completed-Block Routing Ablation

We zero the routing weights assigned to completed block sums, renormalize the remaining sources, and measure the effective rank of h_l over the same input at $t \in \{0.3, 0.5, 0.7\}$ (Figure 15a). At block-entry layers (9, 17, 25), the current partial sum has reset, so removal leaves the input embedding and reduces rank by 82–91%. The effect becomes small after the partial sum has rebuilt within a block. This localizes completed-block reuse to the boundary where it is needed most, without attributing the recovered rank to one operator inside the block.

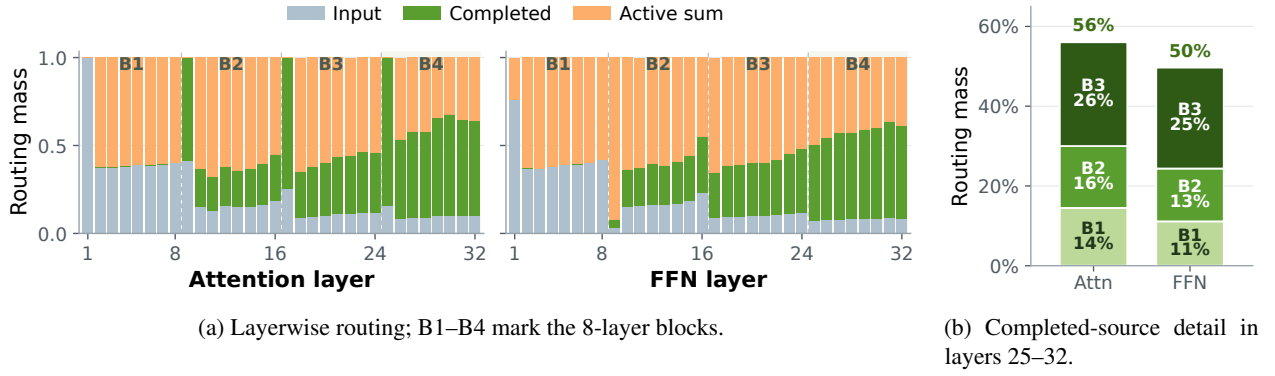


Figure 14 | **AttnRes routing-source composition** (ten-clip mean). In (a), dashed lines mark block resets; the first attention layer carries only the input source.

E.7. Same-Checkpoint Rank Probe

For the gated, RoPE-rotated linear state $S = \sum_n v_n (\beta_n k_n^r)^\top$, we define

$$r_{\text{eff}}(S) = \exp\left(-\sum_i p_i \log p_i\right), \quad p_i = \frac{\sigma_i(S)}{\sum_j \sigma_j(S)}. \quad (5)$$

In implementation, we first average corresponding singular values across attention heads, then normalize the averaged spectrum and compute r_{eff} . We toggle depth aggregation on and off on the same production checkpoint and prompt, pairing noise seeds 0–3 between the two conditions. Figure 6b reports the mean and standard deviation across seeds at each linear layer, and the deep-layer summary averages layers ≥ 15 . No weights or activations other than the AttnRes path are changed, isolating a representation-level effect of the trained router. For context (Figure 15b), we place this result beside a public 2B pure-linear model and a separately trained no-AttnRes model. These differ in width, training horizon, and checkpoint, so the comparison is contextual rather than an AttnRes effect estimate.

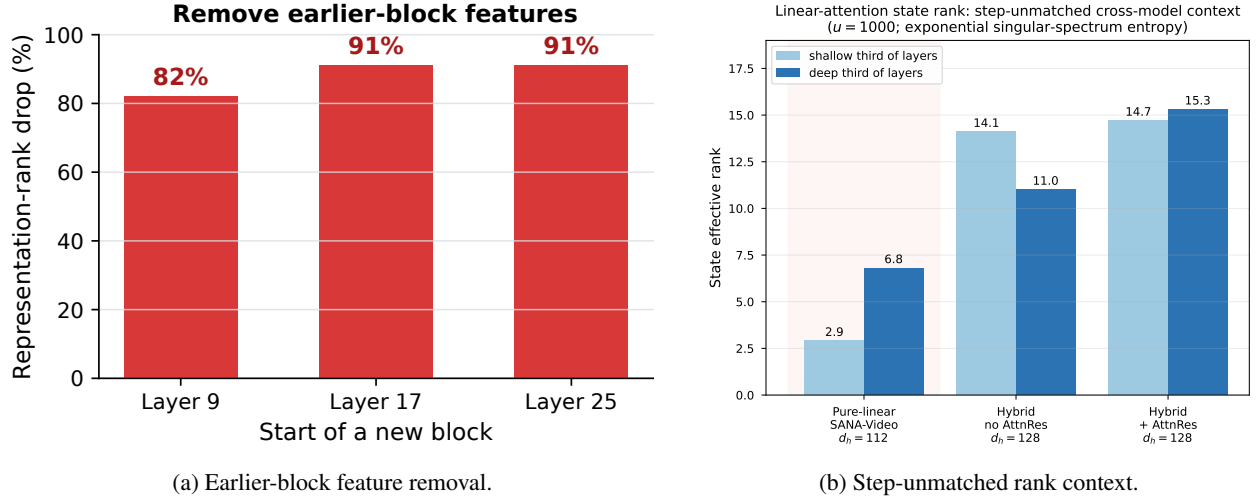


Figure 15 | **Routing intervention and rank context**. (a) Representation-rank drop after removing completed-block sources at block entries. (b) Descriptive comparison across separately trained models that differ in width, training horizon, and checkpoint.

F. Efficiency and Deployment Analysis

This section first expands the architecture-scaling results, then isolates specific costs and deployment levers. Appendix C.2 defines the protocols; results from different subsections are not composed unless a table explicitly reports

the combined measurement.

F.1. Architecture Scaling Across Resolution

Table 9 gives the exact values behind Figure 5(a); every variant runs its best kernels under `torch.compile` (protocol in Appendix C.2).

Table 8 | **Representative compiled profiling results.** Latencies are in ms and Hybrid denotes 25% softmax; Table 9 gives the complete sweep.

Shape	Tokens	Hybrid	Full	Speedup
480p/81f	4,290	82.7	95.6	1.16×
720p/121f	14,720	305.8	474.7	1.55×
1080p/121f	32,640	833.1	1,671.7	2.01×

Table 9 | **Compiled DiT-forward latency across resolutions** (one H100; ms). All variants use their best kernels under the common protocol in Appendix C.2.

Model	480p/81f	480p/121f	720p/81f	720p/121f	1080p/81f	1080p/121f
25% softmax (8/32, default)	82.7	117.7	196.4	305.8	497.0	833.1
50% softmax (1:1)	87.2	126.2	221.8	351.5	626.2	1,075.9
100% softmax (Full)	95.6	145.4	274.5	474.7	878.3	1,671.7
Speedup (100%→25%)	1.16×	1.24×	1.40×	1.55×	1.77×	2.01×

F.2. Model-Size Scaling at Fixed Sequence Length

Table 10 fixes a 720p/10 s sequence while jointly increasing width and depth. Absolute time saved by the 25% hybrid grows from 128 to 948 ms, while Full/Hybrid speedup narrows from 1.88×

Table 10 | **Compiled model-size scaling at fixed 720p/10 s** (19.3K tokens; ms). Matched variants use the common protocol without AttnRes; scale rounds the hybrid parameter count.

Scale (B)	Hidden	Depth	Anchors	Hybrid	Full	Saved	Full/Hybrid
1.2	1,536	24	6	145.2	272.8	127.6	1.88×
2.5	2,048	28	7	258.5	461.4	202.8	1.79×
4.5	2,560	32	8	411.8	690.9	279.1	1.68×
7.2	3,072	36	9	611.0	993.7	382.7	1.63×
10.9	3,584	40	10	886.0	1,391.8	505.8	1.57×
15.7	4,096	44	11	1,197.9	1,823.5	625.6	1.52×
21.6	4,608	48	12	1,615.0	2,396.6	781.7	1.48×
28.9	5,120	52	13	2,094.3	3,042.5	948.2	1.45×

F.3. 14B Cross-Hardware Scaling

Figure 16 and Table 11 compare the same 14B hybrid implementation on H100 and GB200. Across the measured resolution and duration range, GB200 reduces forward latency by 1.66–1.98×

Table 11 | **Formal 14B-configuration H100/GB200 profiling values.** The hybrid has 48 layers at width 3,584 (13.069B); latency is the median compiled DiT-forward time in ms and memory is peak allocated GiB.

Sweep	Decoded shape	Tokens	H100	GB200	Speedup	Peak GiB (H/G)
Resolution	480×832×81	4,290	200.8	109.6	1.83×	25.0 / 25.0
Resolution	480×832×121	6,240	299.6	155.5	1.93×	25.3 / 25.2
Resolution	736×1280×81	10,120	490.7	250.3	1.96×	25.8 / 25.7
Resolution	736×1280×121	14,720	768.5	389.1	1.98×	26.4 / 26.3
Resolution	1088×1920×81	22,440	1,222.5	663.8	1.84×	27.3 / 27.3
Resolution	1088×1920×121	32,640	2,032.4	1,069.5	1.90×	28.5 / 28.5
Duration	736×1280×241	28,520	1,657.0	897.2	1.85×	28.1 / 28.1
Duration	736×1280×361	42,320	2,760.4	1,548.7	1.78×	29.7 / 29.7
Duration	736×1280×481	56,120	4,114.8	2,320.9	1.77×	31.4 / 31.4
Duration	736×1280×721	83,720	7,308.0	4,401.8	1.66×	34.9 / 34.8
Duration	736×1280×1081	125,120	14,204.7	8,569.8	1.66×	40.0 / 40.0

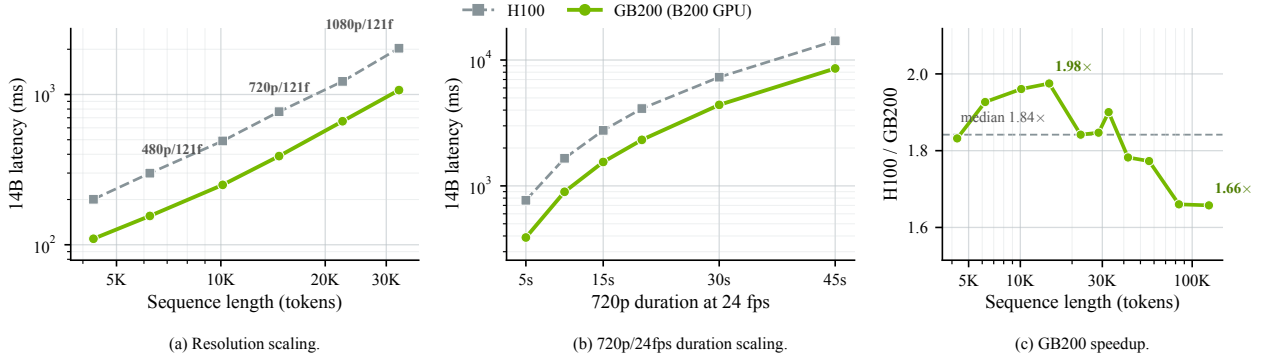


Figure 16 | **Matched 14B cross-hardware DiT-forward profiling** (one GPU, bf16, batch one; AttnRes off). (a) Resolution/frame scaling. (b) 720p duration scaling. (c) H100/GB200 latency ratio. Both devices use identical inputs and compiled kernels through 125.1K tokens.

F.4. 14B Hybrid versus Full-Softmax Scaling

Figure 17 compares parameter-matched hybrid and full-softmax 14B instances. As duration grows from 5 to 45 s, Full/Hybrid speedup rises from 1.47× to 2.81× on H100 and from 1.60× to 3.10× on GB200. The result confirms that the mostly-linear advantage persists at the larger architecture scale and strengthens with sequence length; it remains a systems comparison, not a quality claim.

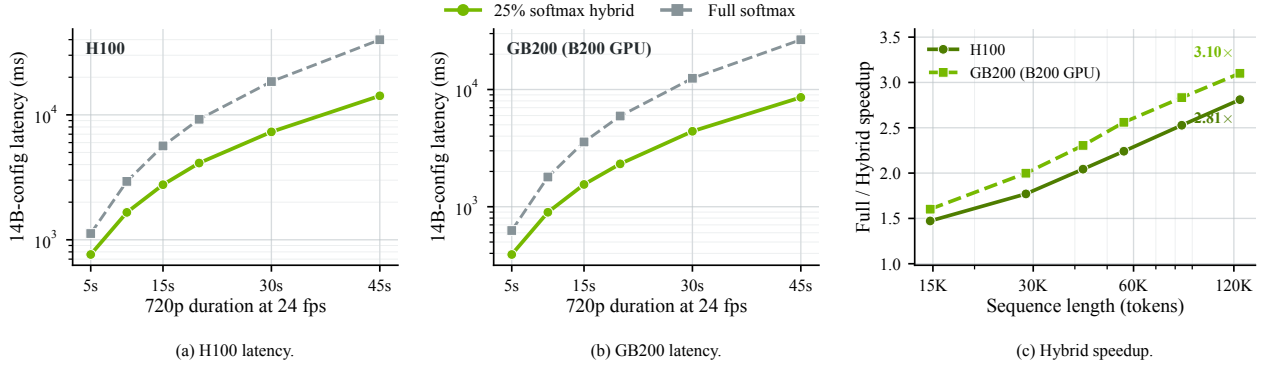


Figure 17 | **Matched 14B hybrid versus full-softmax DiT-forward profiling** (48 layers, width 3,584; one GPU, bf16, batch one; AttnRes off). (a,b) Latency over the 720p/24 fps duration grid on H100 and B200. (c) Full/Hybrid latency ratio. Both layouts use the same width, depth, inputs, Flash SDPA implementation for softmax, and compiled protocol.

F.5. Long-Sequence Tensor-Shape Scaling at 720p

Figure 5(b) and Table 12 are the paper’s duration-scaling result under the standard compiled 24 fps protocol. Full/25%-hybrid speedup grows from 1.55 $\times$ at 5 s to 3.17 $\times$ at 60 s; each variant is measured in an isolated subprocess under the same compile mode.

Table 12 | **Compiled 720p duration scaling** (one H100; ms). Hybrid is 25% softmax; 50% is a separate matched run; Full/Hybrid is speedup.

Duration/frames	Tokens	Hybrid	50%	Full	Speedup
5s/121f	14,720	305.8	351.5	474.7	1.55 $\times$
10s/241f	28,520	675.1	882.3	1,280.0	1.90 $\times$
15s/361f	42,320	1,146.0	1,600.1	2,537.3	2.21 $\times$
20s/481f	56,120	1,720.7	2,530.5	4,140.4	2.41 $\times$
30s/721f	83,720	3,158.2	4,997.0	8,589.0	2.72 $\times$
45s/1081f	125,120	6,040.4	10,098.8	18,147.8	3.00 $\times$
60s/1441f	166,520	10,174.2	17,028.1	32,256.9	3.17 $\times$

F.6. Temporal-Convolution FFN Ablation

SANA-Video uses an additional temporal-convolution path inside its FFN, whereas SANA-Video 2.0 relies on attention for spatiotemporal mixing and keeps a standard SwiGLU FFN. To measure the cost of that choice, we hold depth and width fixed, change only the FFN path, and sweep 720p duration at two model scales (Table 13). The variants are architecture-matched rather than parameter-matched because the temporal convolution adds weights. Its overhead grows with duration and reaches 20–29% at the larger scale. Removing it therefore simplifies the backbone and preserves the scaling benefit of the selected attention ratio.

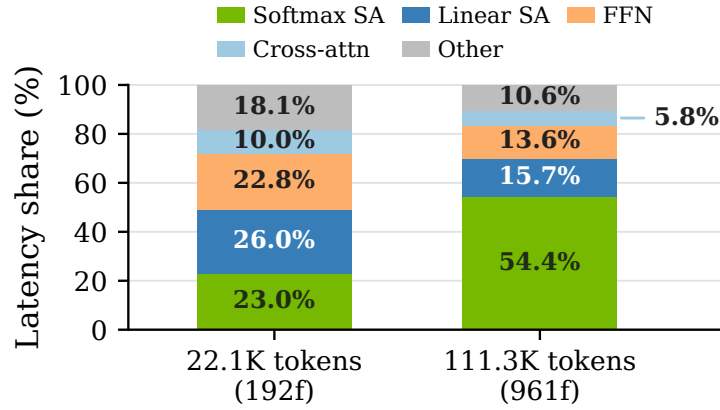
F.7. Module-Level Composition

Figure 18 reports mutually exclusive top-level module shares from an eager shape diagnostic (modules are non-overlapping, so the shares are additive). From 22.1K to 111.3K tokens, the eight softmax anchors grow from 23.0% to 54.4% of forward time, while linear self-attention falls from 26.0% to 15.7%, which motivates anchor-specific kernels for long shapes.

Table 13 | **Controlled FFN latency swap** at 720p (one H100; ms). Temporal convolution adds parameters; Δ is its latency overhead and $\dagger$ marks excluded 60 s values.

Backbone	FFN	5s	15s	20s	30s	45s	60s
$L=20, d=2240$	SwiGLU	52	126	161	242	367	510
	Temporal-conv	64	157	202	302	455	627
	Δ	+12	+31	+42	+60	+88	+117
$L=32, d=2560$	SwiGLU	306	906	1258	2036	3416	4994 $\dagger$
	Temporal-conv	396	1141	1573	2499	4091	10043 $\dagger$
	Δ	+90	+236	+315	+463	+677	— $\dagger$

Figure 18 | **Eager module composition** at 22.1K/111.3K tokens (random initialization).



F.8. Sparse Softmax-Anchor Attention

Only the 25% softmax anchors retain quadratic sequence cost, so they are the natural target for a deployment-only sparse kernel. We use PISA [15], which evaluates selected blocks exactly and approximates the remainder, at 10% exact-block density while keeping the first four anchors dense. Table 14 measures its direct end-to-end contribution. Because this stage changes the attention computation, we report it as an approximate deployment operating point rather than a modeling contribution.

F.9. Compile-Friendly AttnRes

The original inference path mutates a source buffer and reduces a growing prefix, which is difficult for `torch.compile` to fuse. We instead expose the at-most-five sources as a static list and accumulate their weighted sum in fp32 without stacking all sources. Relative to the buffer path, full-forward drift is negligible in fp32 and 0.68% in bf16. The rewrite improves compiled/eager speedup from $1.47\times$ to $2.11\times$ at 8 s and from $1.21\times$ to $1.49\times$ at 60 s. These numbers compare router implementations, not complete model deployments.

F.10. End-to-End Sol-Engine Deployment

We finally test whether the architecture benefits survive a complete deployment stack rather than multiplying isolated microbenchmarks. On one B200 at $736\times 1280\times 193$ (720p, 8 s), Sol-Engine [18] first fuses memory-bound router, RoPE, normalization, gating, and SwiGLU paths; caches step-invariant conditioning; compiles the linear-attention core; and uses exact untiled VAE decoding. EasyCache-style residual reuse [46] then evaluates 33 of 50 denoising steps, and PISA applies 10% exact-block density to the remaining softmax anchors while keeping the first four anchors dense.

Table 14 reports direct cumulative measurements from this single 50-step protocol. Kernel and execution optimiza-

tion reduce latency from 62.65 to 30.74 s; residual reuse reaches 20.89 s; and sparse anchors reach 17.52 s, for a total $3.58\times$ speedup. The first stage is function-preserving, while caching and sparse attention introduce approximation. This B200 end-to-end result is separate from the H100 DiT-forward scaling studies and is not multiplied with them.

Table 14 | **Direct Sol-Engine deployment measurements** on one B200 at $736\times 1280\times 193$ (720p, 8 s). All rows use the same 50-step end-to-end protocol; latency includes denoising and VAE decoding.

Stage	DiT evals	Latency (s)	Speedup
Baseline	50	62.65	$1.00\times$
Kernel and execution optimization	50	30.74	$2.04\times$
+ residual reuse	33	20.89	$3.00\times$
+ sparse anchors	33	17.52	$3.58\times$

G. Extended Related Work

This section expands Section 7 with per-line detail and explicit contrasts to SANA-Video 2.0; the main text gives the condensed version.

G.1. Video Generation Architectures

Most open video DiTs, including Wan 2.1/2.2, HunyuanVideo, CogVideoX, and Open-Sora, use full 3D softmax attention [14, 25, 34, 40]. Their cost grows quadratically with the latent sequence. Other systems control this cost by generating temporal chunks (MAGI-1 [28]) or reducing the token count through stronger VAE compression (LTX-Video [8]). SANA-Video [3] instead uses a pure-linear DiT with temporal convolution for local mixing. We follow the operator-level route but remove that convolution (Appendix F.6) and use periodic softmax anchors to restore direct token interactions.

G.2. Linear, State-Space, and Hybrid Attention

Linear attention and state-space models replace the $N\times N$ attention map with a fixed-size state [1, 7, 10, 16, 22, 32, 39]. This gives favorable scaling but can restrict representation rank [4]. Hybrid language models therefore retain a minority of softmax layers: Qwen3-Next and Kimi-Linear use a 3:1 linear-to-softmax layout, while Kimi K3 combines a related layout with Attention Residuals [11, 12, 13, 27]. Our setting differs in two ways. Video diffusion is bidirectional rather than autoregressive, and the model is trained from scratch rather than converted from a pretrained softmax backbone. We also adapt depth routing by sharing its query across layers and removing the explicit router-timestep input that our ablations find redundant with the timestep-conditioned DiT features. DiG [48] and Attention Surgery [6] provide complementary image and post-hoc video linearization settings.

G.3. Sparse and Approximate Softmax Attention

Sparse-VideoGen, Radial Attention, SpargeAttn, PISA, and Native Sparse Attention reduce the work inside each softmax operator [15, 17, 36, 42, 44]. Our hybrid instead reduces how many layers use softmax. The two ideas are complementary: in deployment, sparsity is applied only to the fixed softmax anchors while the linear majority is unchanged (Appendix F.8).